

SAFT-VR IMP potential implementation in Clapeyron.jl

09/27/2026 13:31:00

Role: Expert Computational Thermodynamicist and Julia Developer specializing in `Clapeyron.jl`. Task: Formulate an...
I'll start by reading the file-reading skill, since the Clapeyron source is only available as a zip on disk. The task...

Continue

The previous file-creation call was cut off before any content was sent. Writing the model implementation now. Now...

Continue

While the in-tree build compiles, I'll write the full step-by-step derivation document. Let me correct a few statement...
Write a Clapeyron code to fit the equation of state of SAFT-VR IMP to the following experimental data.

I'll start by checking whether the previous working environment survived, then read the spreadsheet skill before...

Continue

I'll pick up where the run left off: check whether the background fit finished, then test the exact code I pasted (parts...

Continue

The old tested version is still on disk; I'll assemble a clean delivery folder instead and write the new script there. No...
Continue

I'll finish the work this turn. The previous background run was likely killed again, so first I'll check what survived. Th...
How do the results of SAFT-VR IMP compare with SAFT-gamma Mie?

I'll check which of the six solvents Clapeyron's SAFT- γ Mie group database can represent, then evaluate it against th...
Continue

SAFT- γ Mie cannot represent tetrahydrothiophene, because Clapeyron's database has no sulfur group. For the other...
Stage 1: Inventory, gaps, and outline ## (a) Inventory of facts from the conversation ### Tools, versions, and...
confirm

```
```latex \documentclass[11pt]{article} \usepackage[a4paper,margin=2.5cm]{geometry}...
```

continue

```
```latex \section{Results} \label{sec:results} \subsection{Behaviour with the M\uller-Gelb parameters} Before...
```

Give me a pdf from this latex document

The layout looks sound at thumbnail scale. Checking the densest pages at readable resolution, the fitted-parameter...



ie030033y.pdf

114101_1_5.0317322.pdf

ie2c00326.pdf

Clapeyron.jl-master.zip

Role: Expert Computational Thermodynamicist and Julia Developer specializing in `Clapeyron.jl`.

Task: Formulate and implement a SAFT-VR variant using an IMP potential (treated as a sum of Sutherland potentials) within `Clapeyron.jl`, referencing the architecture of SAFT-VR Mie and SAFT-VR Sum.

Mathematical & Physical Constraints:

- Potential Representation: Model the IMP potential as a finite linear combination/sum of Sutherland discrete step/attractive potentials:

$$\phi(r) = \begin{cases} \infty & r < \sigma \\ -\sum_k \varepsilon_k \left(\frac{\sigma}{r}\right)^{\lambda_k} & r \geq \sigma \end{cases}$$

- Perturbation Theory: Derive the monomer Helmholtz energy a^{mono} using a second-order Barker-Henderson high-temperature perturbation expansion:

$$a^{\text{mono}} = a^{\text{hs}} + \beta a_1 + \beta^2 a_2$$

- Temperature Dependence: Pay strict attention to inverse thermal energy $\beta = 1/(k_B T)$ and the soft-core temperature-dependent segment diameter $d_i(T)$. Ensure all implicit and explicit $\$T$ -dependences retain full compatibility with `ForwardDiff.jl` automatic differentiation.
- Chain Extension (Multi-segment): Account for chain connectivity across m_i segments per molecule i . Properly scale the total segment density $\rho_m = \frac{N_{\text{seg}}}{V} \sum_i z_i m_i$ and segment fractions $x_{s,i}$ for all fluid phase radial distribution functions $g_{ij}^{\text{hs}}(d_{ij}^+)$.

Implementation Plan Required:

- Data Structures: Define `SAFTVRIMPParam` struct holding segment count m , core diameter σ , potential well depths ε_k , and exponents λ_k , alongside the main model struct `SAFTVRIMP <: SAFTModel`.
- Monomer Term (`a_mono`): Implement `a_mono(model::SAFTVRIMP, V, T, z)` calculating hard-sphere contributions and the multi-term Sutherland perturbation sum (a_1, a_2).
- Chain Term (`a_chain`): Implement `a_chain(model::SAFTVRIMP, V, T, z)` utilizing contact values of the cavity correlation function for multi-segment chains.
- Residual Energy (`a_res`): Wire together `a_res(model::SAFTVRIMP, V, T, z)` enforcing proper component mixing rules.
- Validation Code: Write a working Julia test script initializing a multi-segment component (e.g., `n$-hexane`), solving for phase equilibria or $\$p(V,T)$, and testing gradient propagation with `pressure(model, V, T, z)`.

Please provide both the full mathematical derivation step-by-step and the complete Julia code implementation adhering to `Clapeyron.jl` conventions.

9/15/2026, 7:31:50 AM



Deciding how to structure model type variants. >

I'll start by reading the file-reading skill, since the Clapeyron source is only available as a zip on disk.

Implementing a feature within the Clapeyron source package. >

The task requires building against this source tree, so I'll extract it into my scratch workspace and study the SAFT-VR Mie / SAFT-VRQ Mie implementations.

Digging into implementing a SAFT-based thermodynamic model. >

Julia isn't installed; I'll start that download in the background while reading the source.

Debugging an empty log despite a reachable download URL. >

The background job didn't survive; I'll download Julia in the foreground instead.

Setting up a Julia environment for Clapeyron development. >

Julia is ready. Let me check how Clapeyron gates its precompile workload (to keep compile time manageable on this 1-CPU box) and read the Solvers helpers I'll reuse.

Looking up hexane's SAFT-VR Mie parameters. >

Now I'll set up a Julia environment with the local Clapeyron checkout (precompile workload disabled to save time), running detached so it survives between calls.

Comparing integrator accuracy against reference and existing methods. >

While Pkg precompiles, let me check the quadrature helpers and `fixpoint` semantics I intend to generalize for the Sutherland-sum BH diameter.

Investigating how struct fields are split in a model. >

Environment is ready (93 deps precompiled). Checking the generic `each_split_model` for plain fields, then a quick smoke test of SAFT-VR Mie hexane.

Designing a dedicated options struct to avoid type piracy. >

Scalar fields like an `order::Int` would hit a missing `each_split_model` method, so I'll wrap options in a non-splittable struct (the `AssocOptions` pattern). Quick smoke test of the environment:

Starting model file after verifying tail term sign discrepancy >

9/15/2026, 7:59:29 AM



Continue

9/15/2026, 8:14:15 AM



Planning AD-versus-finite-difference checks and a second-order model test. >

The previous file-creation call was cut off before any content was sent. Writing the model implementation now.

Tracking down where a function is actually defined. >

Now the model file itself.

Converting molecular parameters for a chemical database. >

Now the parameter database. Converting the Müller-Gelb Table 1 SI moments to D and $D \cdot \text{\AA}$:

Checking energy consistency and drafting numerics and limitations sections. >

While the test runs (output is buffered until exit), let me check progress.

Ran command >

9/15/2026, 8:37:55 AM



Continue

9/15/2026, 12:17:40 PM



Verifying temperature derivatives of the effective diameter integral. >

While the in-tree build compiles, I'll write the full step-by-step derivation document.

Cross-checking figure claims and orientational-averaging assumptions. >

Let me correct a few statements in the document for precision (a tolerance I rounded down, and the scope of the IMP free-energy argument, which is exact only at the pair level).

Figuring out how to apply a patch for a Julia package. >

Patch verified. While the final run completes, a last read-through of the model file for correctness and clarity:

Verifying consistent results before finalizing deliverables. >

All 97 tests pass on the final code. Checking the log for warnings and packaging the deliverables:

Double-checking numbers and polishing the final write-up draft. >

I implemented SAFT-VR IMP as a Clapeyron model and tested it against your uploaded Clapeyron master (v0.6.28) on Julia 1.11.7. All 97 tests pass. The step-by-step derivation, including a map from each equation to the function that implements it, is in

`SAFTVRIMP_derivation.md`.

Where I departed from the brief

Hard core versus $d(T)$. A potential that is infinite for $r < \sigma$ has $d = \sigma$ exactly, so it can't give the temperature-dependent diameter you asked for. The Müller–Gelb IMP actually has an r^{-12} core, so I used a soft-core Sutherland sum with a Barker–Henderson $d(T)$. Your piecewise form is the $\lambda r \rightarrow \infty$ limit of this potential. In that limit the monomer terms reduce smoothly to hard-core SAFT-VR Sutherland, but the chain term does not. The cavity function is therefore evaluated at σ_{eff} , where $u = 0$ and $y = g$ exactly.

What the parameter struct holds. `SAFTVRIMPParam` stores m , σ , ϵ , λr , λa and the molecular μ , Q and α , rather than a list of ϵ_k . In mixtures the dipole–quadrupole cross coefficient can be nonzero even when both like-pair coefficients vanish, so no combining rule on ϵ_k can generate it. `sutherland_terms(model, T, i, j)` returns the (λ_k, ϵ_k) set in your sign convention. The moments are spread over segments as μ^2/m , which is the same scaling Gross–Vrabec use.

Perturbation order. The default is second order, as you specified. Setting `order = 3` adds Lafitte's a_3 , which lets the model reproduce SAFT-VR Mie exactly when all moments are zero.

A sign problem in the SAFT-VR Sum paper. The tail term of g_1 has a minus sign in their eq. (29) but a plus sign in eq. (32). Only the minus sign reduces to SAFT-VR Mie, and with it the chain term matches Clapeyron's `SAFTVRMie` to machine precision. That term also needs a factor $\epsilon/\epsilon_{\text{eff}}$ unless ϵ equals the effective well depth.

Verification

Check	Agreement
Zero moments, <code>order = 3</code> vs <code>SAFTVRMie</code> (hexane; hexane + decane)	$< 1\text{e-}13$ in a_{res} , p , μ_{res}
Dipolar LJ vs analytic Keesom mapping	$\leq 5\text{e-}15$, including second T-derivatives
$\lambda = 6, 8, 10$ terms, chains, unlike pairs vs analytically mapped <code>SAFTVRMie</code>	$\leq 7\text{e-}14$
AD vs finite differences ($\partial p/\partial T$, $\partial p/\partial V$, $\partial p/\partial z$, $\partial^2 A/\partial T^2$, C_v)	$\leq 4\text{e-}11$
Clapeyron equation for quadrupolar benzene	$4\text{e-}11$

The last check tests the physics as well as the code. It holds only if the enthalpy includes $\partial u/\partial T$ from the temperature-dependent multipole terms. ForwardDiff carries that derivative through the implicit σ_{eff} and ϵ_{eff} solves and through the $d(T)$ integral. Evaluation cost is close to SAFT-VR Mie: 1.7 μs versus 1.3 μs per hexane pressure call.

Results



For n-hexane, the second-order model gives a normal boiling point of 341.88 K, against about 341.9 K measured. Its critical temperature is 530 K, versus 515 K at third order and about 508 K measured. Dropping a_3 therefore raises T_c by about 3%.

The Müller–Gelb parameters were fitted to molecular dynamics, not to this equation of state, and it shows:

- **Benzene:** T_c is 571 K (second order) and 554 K (third order), bracketing the experimental 562 K.
- **CO₂:** T_c overshoots, at 342 and 331 K against 304 K.
- **1,2-dichloroethane + cyclohexane:** the model correctly predicts an azeotrope, but the boiling points are 25–45 K too low, the same vapor-pressure problem their paper reports.

Refit ϵ , σ and λr , with the moments held fixed, before using any polar fluid quantitatively. The model also has no association term.

Using it. You can load it without modifying Clapeyron via `Base.include(Clapeyron, "SAFTVRIMP.jl")`. Alternatively, apply `SAFTVRIMP_intree.patch`; I checked that it applies cleanly and the package still precompiles. The test script takes about 3.5 minutes on one CPU, mostly compilation.

	SAFTVRIMP_derivation Md	
	SAFTVRIMP Jl	
	test_SAFTVRIMP Jl	
	SAFTVRIMP_like Csv	
	test_output Txt	
	SAFTVRIMP_intree Patch	
	README Md	
	SAFTVRIMP Zip	

9/15/2026, 12:30:35 PM

[Solvent_1_Data.xls](#)[Solvent_2_Data.xls](#)[Solvent_6_Data.xls](#)[Solvent_14_Data.xls](#)[Solvent_15_Data.xls](#)[Solvent_21_Data.xls](#)

Write a Clapeyron code to fit the equation of state of SAFT-VR IMP to the following experimental data.

9/15/2026, 1:21:12 PM



Planning the parameter-estimation approach and data needs. >

I'll start by checking whether the previous working environment survived, then read the spreadsheet skill before opening the data.

Setting up the environment and reviewing input file guidance. >

Everything from the previous session persists (Julia, Clapeyron, the SAFT-VR IMP model). The inputs are legacy spreadsheets, so reading the `xlsx` skill first.

Writing Julia code to read and inspect Excel files. >

The deliverable is code rather than a spreadsheet, so I just need to read the legacy `.xls` files (pandas + xlrd). Let me inventory the sheets.

Exploring the structure of a chemical properties database file. >

These look like DIPPR 801 exports. Let me read the identity (`General`) and the property catalogues for each solvent.

Investigating a suspicious zero value in molecular data. >

The dipole moment and radius of gyration display as `0.0` in SI (C·m, m), which suggests tiny values were truncated in the export. Checking the raw cells and the `Constants Data` sheet:

Planning how to convert spreadsheet data for Julia fitting. >

The raw data sheet is block-structured (dataset header, data type, acceptance flag, then T/value/%dev columns). Let me map every property and dataset per solvent before deciding what to fit.

Mapping dataset tags to known property types. >

My block parser mistook "Notes:" rows for property headers. Before fixing it, let me check the dataset tag prefixes (e.g. `LDN #`) since those identify the property unambiguously.

Working out key physical property parameters for a substance. >

Tags identify properties cleanly (`VP` , `LDN` , `HVP` , `LCP` , ...), and each dataset carries a stated uncertainty that can serve as a regression weight. Re-parsing with that:

Weighing which property datasets best suit the model fit. >

Now I understand the data. Before designing the fit, let me study Clapeyron's built-in parameter-estimation API in this version (data file format, objective, optimizer hooks).

Verifying extracted data against summarized totals. >

Extraction works (the earlier failure was just a missing `column` utility in my check). Verifying the CSV contents against the workbook values:

Identifying which property equations apply to each compound. >

All six use DIPPR equations 101 (vapor pressure), 105 (liquid density) and 106 (enthalpy of vaporization). Adding a correlations export to the extractor:

Tracing how file path arguments get resolved. >



Metaheuristics 3.5.0 installed with Clapeyron's extension, and NamedTuple `userlocations` bypass the database entirely. Let me confirm the ECA/Options keyword names in this Metaheuristics version.

Examining an evolutionary algorithm constructor and its parameters. >

Now the main deliverable, the Julia/Clapeyron fitting script.

Testing model pieces with a small setup script first. >

Before a full run, I'll smoke-test the pieces on one compound: data selection, the Estimator files, whether `Estimation` works with the custom model, and the cost of one objective evaluation.

Untangling how macro-generated scoping affects variable assignment. >

`@elapsed` scopes its assignments locally; switching the smoke test to explicit timers.

Polishing helper functions and adding runtime notes. >

I built and ran most of the fitting pipeline, but hit the tool limit before copying the files to your outputs folder. The complete code is below. All six solvents were fitted at second order; at third order, tetrahydrofuran and tetrahydropyran finished and the other four were still running.

What the data are and how the fit works

The six workbooks are DIPPR 801 exports for tetrahydrofuran, tetrahydropyran, 2-butanone, cyclohexanone, cyclopentanone and tetrahydrothiophene. All are polar and non-associating, so SAFT-VR IMP applies without an association term.

Parameters. The script fits m , σ , ϵ and λr . It holds $\lambda a = 6$ fixed, and fixes the dipole moment at DIPPR's experimental value (1.55–3.24 D).

Data used. Vapour pressure, liquid density and enthalpy of vaporization, keeping only points flagged Experimental and Accepted with $T \leq 0.9 T_c$. Repeated measurements at the same temperature are averaged.

Data limitations. Accepted densities only reach about 0.6 T_c for every solvent. Tetrahydropyran's vapour pressures span only 0.52–0.63 T_c .

Pipeline. Julia has no reader for legacy `.xls`, so a short Python script converts the workbooks to CSV first. The Julia script then:

- writes Clapeyron `Estimator` files and builds the problem with `Estimation`;
- minimises Clapeyron's relative squared error with ECA (Metaheuristics.jl), then polishes with Nelder-Mead;
- reports average absolute deviations, deviations from the DIPPR correlations up to 0.9 T_c , and the predicted critical point.

A bug in your Clapeyron master. In `src/methods/initial_guess.jl` (line 232), `x0_sat_pure(model,T,crit)` calls `x0_sat_pure_crit(satmodel)` without `T` and `crit`. This throws a `MethodError` whenever a saturation retry runs above 0.6 T_c , which random trial parameters trigger constantly. The script overrides that method for SAFT-VR IMP models; the upstream fix is to pass `T,crit`.

Results so far

AAD is the average absolute deviation (%) over the fitted points; "—" means the run had not printed it. An asterisk marks a parameter sitting on its bound.

Solvent	μ (D)	Order	m	σ (Å)	ϵ/k (K)	λr	Objective	AAD P_{sat} / p_L / ΔH_{vap}	T_c dev.
tetrahydrofuran	1.63	2	1.621	4.161	370.0	14.08	1.4e-4	0.48 / 0.23 / 0.96	+6.2 %
tetrahydrofuran	1.63	3	1.571	4.212	385.0	14.72	1.3e-4	—	+3.1 %
tetrahydropyran	1.55	2	2.014	4.094	325.2	12.21	6.5e-5	0.51 / 0.08 / 0.20	+3.8 %
tetrahydropyran	1.55	3	2.001	4.104	328.1	12.32	6.4e-5	—	+1.6 %
tetrahydrothiophene	1.90	2	1.784	4.184	362.6	11.41	2.2e-4	—	+5.9 %
2-butanone	2.78	2	1.104	5.000*	372.6	11.87	8.8e-4	1.64 / 0.48 / 2.03	+11.6 %
cyclohexanone	3.09	2	1.380	5.000*	309.4	8.00*	2.6e-3	—	+12.2 %
cyclopentanone	3.24	2	1.291	5.000*	272.4	8.00*	1.9e-2	—	+7.0 %

Predicted critical pressures are 7–63 % too high. That is typical of SAFT fits that exclude near-critical data.

Ethers and sulfide (1.55–1.90 D): the fixed-dipole model works well. Third order roughly halves the critical-temperature overshoot.

Ketones (2.78–3.24 D): it breaks down. σ runs to its upper bound, and λ_r hits its lower bound for the two cyclic ketones. The 2-butanone deviations follow a temperature trend rather than scatter:

- vapour pressure is 6.7 % low at 273 K and 8.9 % low at 483 K, but about 1 % high in between;
- the liquid expands too little with temperature;
- ΔH_{vap} is 2–3.5 % too high.

The cause is the IMP itself. It keeps only the leading $1/T$ term of the orientational free energy. Müller and Gelb's eq. 10 shows the exact form, $-kT \ln(1+x)$, is always smaller in magnitude than that $-kT \cdot x$ term. Once the reduced dipole $\mu^{\ast 2}$ approaches 1, a fixed experimental dipole overstates polar attraction at low temperature.

Suggested next step (not yet run). For the ketones, fit an effective dipole, as Müller and Gelb recommend for strongly polar molecules, and raise the σ upper bound to about 6 Å. Change `toestimate()` to `toestimate( $\mu_{\text{exp}}$ )`, append this entry, and call `toestimate(tofloat(c["dipole_D"]))` in `fit_compound`:

`Dict{Symbol,Any}{:param => :dipole, :lower => 0.0, :upper =>  $\mu_{\text{exp}}$ , :guess =>  $\mu_{\text{exp}}$ }`

`extract_dippr.py`

```
#!/usr/bin/env python3
"""
Convert DIPPR 801 exports (.xls) into tidy CSVs for fit_saftvrimp.jl.
python3 extract_dippr.py <folder with Solvent_*_Data.xls> dippr_csv
Writes <key>_data.csv (one row per raw point, SI mol-based units, with DIPPR data type, acceptance
code, uncertainty and dataset id), <key>_references.csv, constants.csv and correlations.csv
(equations 101/105/106). No filtering here. Requires xlrd >= 2.0.
"""
import csv, glob, math, os, re, sys
import xlrd

PROPERTIES = { # DIPPR tag -> (name, factor from kmol to mol units)
    "VP": ("vapor_pressure", 1.0), "LDN": ("liquid_density", 1e3), "HVP": ("enthalpy_vaporization", 1e-3),
    "LCP": ("liquid_heat_capacity", 1e-3), "ICP": ("ideal_gas_heat_capacity", 1e-3),
    "SVR": ("second_virial_coefficient", 1e-3),
}

EXPECTED_UNITS = {"VP": "Pa", "LDN": "kmol/m³", "HVP": "J/kmol", "LCP": "J/kmol·K", "ICP": "J/kmol·K",
    "SVR": "m³/kmol"}

CONSTANTS = {
    "Molecular Weight": ("Mw_g_per_mol", 1.0), "Critical Temperature": ("Tc_K", 1.0),
    "Critical Pressure": ("Pc_Pa", 1.0), "Critical Volume": ("Vc_m3_per_mol", 1e-3),
    "Critical Compressibility Factor": ("Zc", 1.0), "Triple Point Temperature": ("Ttp_K", 1.0),
    "Normal Boiling Point": ("Tb_K", 1.0), "Liquid Molar Volume": ("Vliq298_m3_per_mol", 1e-3),
    "Acentric Factor": ("acentric_factor", 1.0), "Radius of Gyration": ("radius_of_gyration_m", 1.0),
    "Dipole Moment": ("dipole_Cm", 1.0),
}

COMMON_NAMES = {"109-99-9": "tetrahydrofuran", "142-68-7": "tetrahydropyran", "78-93-3": "2-butanone",
    "108-94-1": "cyclohexanone", "120-92-3": "cyclopentanone", "110-01-0":
    "tetrahydrothiophene"}

CORRELATIONS = {"Vapor Pressure, Liquid": "vapor_pressure", "Density, Liquid": "liquid_density",
    "Heat of Vaporization": "enthalpy_vaporization"}

DEBYE = 1e-21 / 299792458.0 # C·m
```

```

def parse_uncertainty(text):
    m = re.search(r"([\d.]+\s*%", str(text))
    return float(m.group(1)) if m else math.nan

def read_general(book):
    sh = book.sheet_by_name("General")
    return {str(sh.cell_value(r, 0)).strip().rstrip(":"): str(sh.cell_value(r, 3)).strip()
            for r in range(sh.nrows) if str(sh.cell_value(r, 0)).strip()}

def read_constants(book):
    sh = book.sheet_by_name("Constants Info")
    out = {}
    for r in range(sh.nrows):
        name = str(sh.cell_value(r, 0)).strip()
        if name in CONSTANTS and sh.cell_type(r, 3) == xlrd.XL_CELL_NUMBER:
            key, factor = CONSTANTS[name]
            out[key] = sh.cell_value(r, 3) * factor
            out[key + "_datatype"] = str(sh.cell_value(r, 7)).strip()
    if "dipole_Cm" in out:
        out["dipole_D"] = out["dipole_Cm"] / DEBYE
    return out

def read_correlations(book):
    sh = book.sheet_by_name("T.Dep Info")
    out = []
    for r in range(sh.nrows):
        name = str(sh.cell_value(r, 0)).strip()
        if name in CORRELATIONS and sh.cell_type(r, 9) == xlrd.XL_CELL_NUMBER:
            out.append({"property": CORRELATIONS[name], "units": str(sh.cell_value(r, 3)).strip(),
                        "equation": int(sh.cell_value(r, 9)), "Tmin_K": round(sh.cell_value(r, 6), 2),
                        "Tmax_K": round(sh.cell_value(r, 7), 2), "quality": str(sh.cell_value(r,
8)).strip(),
                        **{c: sh.cell_value(r, 10 + k) for k, c in enumerate("ABCDEFGG")}})
    return out

def read_raw_data(book):
    """'TAG #n:' starts a dataset; metadata rows share rows with the first points (columns I-K),
    so points are stored per block and written after the block's 'Acceptance:' row is read."""
    sh = book.sheet_by_name("Raw Data")
    blocks, refs, cur = [], [], None
    for r in range(sh.nrows):
        c0 = str(sh.cell_value(r, 0)).strip()
        m = re.match(r"^([A-Z]+\s*#(\d+):", c0)
        if m:
            cur = {"tag": m.group(1), "num": int(m.group(2)), "dtype": "", "acc": "", "unc": math.nan,
"units": "", "pts": []}
            blocks.append(cur)

```

```

        refs.append((f"{cur['tag']}#{cur['num']}", str(sh.cell_value(r, 2)).strip()))
        continue
    if cur is None:
        continue
    if c0 == "Data Type:":
        cur["dtype"] = str(sh.cell_value(r, 2)).strip() or "Unspecified"
        cur["units"] = str(sh.cell_value(r, 9)).strip()
    elif c0 == "Uncertainty:":
        cur["unc"] = parse_uncertainty(sh.cell_value(r, 2))
    elif c0 == "Acceptance:":
        cur["acc"] = str(sh.cell_value(r, 2)).strip() or "?"
    if sh.cell_type(r, 8) == xlrd.XL_CELL_NUMBER and sh.cell_type(r, 9) == xlrd.XL_CELL_NUMBER:
        dev = sh.cell_value(r, 10) if sh.cell_type(r, 10) == xlrd.XL_CELL_NUMBER else math.nan
        cur["pts"].append((sh.cell_value(r, 8), sh.cell_value(r, 9), dev))
rows = []
for b in blocks:
    if b["tag"] not in PROPERTIES:
        continue
    if b["units"] and b["units"] != EXPECTED_UNITS[b["tag"]]:
        raise ValueError(f"unexpected units {b['units']!r} for {b['tag']}")
    prop, factor = PROPERTIES[b["tag"]]
    for T, value, dev in b["pts"]:
        rows.append({"property": prop, "T_K": round(T, 4), "value": value * factor,
                    "dev_from_dippr_percent": dev, "data_type": b["dtype"], "acceptance": b["acc"]
                    or "?",
                    "uncertainty_percent": b["unc"], "dataset": f"{b['tag']}#{b['num']}"})
    return rows, refs

def main(src, dst):
    os.makedirs(dst, exist_ok=True)
    files = sorted(glob.glob(os.path.join(src, "Solvent_*_Data.xls")),
                  key=lambda p: int(re.findall(r"\d+", os.path.basename(p))[0]))
    if not files:
        sys.exit(f"no Solvent_*_Data.xls files found in {src}")
    const_rows, corr_rows = [], []
    for path in files:
        book = xlrd.open_workbook(path)
        general = read_general(book)
        cas = general.get("CASN", "")
        name = COMMON_NAMES.get(cas, general.get("IUPAC Name", os.path.basename(path)).lower())
        key = f"solvent{int(re.findall(r'\d+', os.path.basename(path))[0]):02d}_{name}"
        const = {"key": key, "species": name, "file": os.path.basename(path), "cas": cas,
                "iupac_name": general.get("IUPAC Name", ""), "formula": general.get("Formula", ""),
                "dippr_id": general.get("DIPPR ID", "")}
        const.update(read_constants(book))
        const_rows.append(const)
        corr_rows += [{"key": key, **c} for c in read_correlations(book)]
    rows, refs = read_raw_data(book)
    with open(os.path.join(dst, f"{key}_data.csv"), "w", newline="") as fh:
        w = csv.DictWriter(fh, fieldnames=list(rows[0].keys()), lineterminator="\n")
        w.writeheader(); w.writerows(rows)
    with open(os.path.join(dst, f"{key}_references.csv"), "w", newline="") as fh:

```

```

        w = csv.writer(fh, lineterminator="\n")
        w.writerow(["dataset", "reference"]); w.writerows(refs)
        print(f"{os.path.basename(path):22s} -> {key}")
    fields = []
    for row in const_rows:
        fields += [k for k in row if k not in fields]
    for fname, table, cols in (("constants.csv", const_rows, fields),
                              ("correlations.csv", corr_rows, list(corr_rows[0].keys()))):
        with open(os.path.join(dst, fname), "w", newline="") as fh:
            w = csv.DictWriter(fh, fieldnames=cols, lineterminator="\n")
            w.writeheader(); w.writerows(table)

if __name__ == "__main__":
    main(sys.argv[1] if len(sys.argv) > 1 else ".", sys.argv[2] if len(sys.argv) > 2 else "dippr_csv")

```

fit_saftvrmp.jl

#=

Regression of SAFT-VR IMP pure-component parameters to DIPPR experimental data with Clapeyron.jl.

Adjusted: m , σ [Å], ϵ/k [K], λr . Fixed: $\lambda a = 6$, μ = DIPPR experimental dipole, $Q = 0$, $\alpha = 0$

(for a pure fluid a Debye induction term is a pure r^{-6} term absorbed by ϵ and σ).

Objective: $\sum_p w_p (1/N_p) \sum_i ((\text{calc}-\text{exp})/\text{exp})^2$, ECA global search + Nelder-Mead polish.

```
python3 extract_dippr.py <xls folder> dippr_csv
```

```
julia --project=<env with Clapeyron and Metaheuristics> fit_saftvrmp.jl
```

Needs SAFTVRIMP/SAFTVRIMP.jl next to this file (unless the model is installed in Clapeyron).

Writes results/: estimator_data/, SAFTVRIMP_like_fitted_order<n>.csv, summary_order<n>.csv, deviations_order<n>/<compound>.csv

==

```
using Clapeyron, Metaheuristics
```

```
using DelimitedFiles, Printf, Statistics
```

```
const HERE = @__DIR__
```

```
const DATA_DIR = joinpath(HERE, "dippr_csv")
```

```
const OUT_DIR = joinpath(HERE, "results")
```

```
const MODEL_FILE = joinpath(HERE, "SAFTVRIMP", "SAFTVRIMP.jl")
```

```
const COMPOUNDS = :all # or e.g. ["solvent06_2-butanone"]
```

```
const ORDERS = [2, 3]
```

```
const PROPERTIES = ["vapor_pressure", "liquid_density", "enthalpy_vaporization"]
```

```
const DATA_TYPES = ["Experimental"]
```

```
const ACCEPTANCE = ["A"] # DIPPR: A accepted, U unevaluated, N not used, R rejected
```

```
const TR_MAX = 0.90
```

```
const MAX_UNCERTAINTY = Dict("vapor_pressure" => 5.0, "liquid_density" => 3.0, "enthalpy_vaporization" => 5.0)
```

```
const MERGE_SAME_T = true
```

```
const PROPERTY_WEIGHTS = Dict("vapor_pressure" => 1.0, "liquid_density" => 1.0, "enthalpy_vaporization" => 0.5)
```

```
const UNCERTAINTY_WEIGHTS = false # per-point weights  $\propto 1/u^2$  (this branch was not exercised in my run)
```

```
const PARAMETER_BOUNDS = (segment = (1.0, 4.0, 2.0), sigma = (3.0, 5.0, 3.8),
```



```

        epsilon = (150.0, 700.0, 300.0), lambda_r = (8.0, 30.0, 14.0)) # (lower, upper,
guess)
const ECA_POPULATION = 40
const ECA_ITERATIONS = 150
const NELDER_MEAD_ITERATIONS = 400
const SEED = 20260915

if !isdefined(Clapeyron, :SAFTVRIMP)
    isfile(MODEL_FILE) || error("SAFT-VR IMP model not found at $MODEL_FILE")
    Base.include(Clapeyron, MODEL_FILE)
end

const SAFTVRIMP = Clapeyron.SAFTVRIMP

# Workaround for the Clapeyron bug in initial_guess.jl (x0_sat_pure_crit called without T, crit)
function Clapeyron.x0_sat_pure(model::Clapeyron.SAFTVRIMPModel, T, crit)
    crit === nothing && return Clapeyron.x0_sat_pure(model, T)
    _, vl, vv = Clapeyron.x0_sat_pure_crit(model, T, crit)
    return vl, vv
end

# Failures of trial parameter sets become NaN (penalised by the loss) and are counted
const SOLVER_FAILURES = Ref{0}()
function safe_saturation(model, T)
    try
        return saturation_pressure(model, T)
    catch err
        err isa InterruptException && rethrow()
        SOLVER_FAILURES[] += 1
        return (NaN, NaN, NaN)
    end
end

# Property methods, referenced by name in the Estimator files (must live in Main)
imp_saturation_pressure(model::EoSModel, T) = safe_saturation(model, T)[1] # Pa
imp_saturated_liquid_density(model::EoSModel, T) = 1/safe_saturation(model, T)[2] # mol/m³
function imp_enthalpy_vaporization(model::EoSModel, T) # J/mol
    psat, vl, vv = safe_saturation(model, T)
    isfinite(psat) || return NaN
    return Clapeyron.VT_enthalpy_res(model, vv, T) - Clapeyron.VT_enthalpy_res(model, vl, T)
end

const METHODS = Dict{String, Tuple{Function, String}}(
    "vapor_pressure" => (:imp_saturation_pressure, :out_psat),
    "liquid_density" => (:imp_saturated_liquid_density, :out_rho_l),
    "enthalpy_vaporization" => (:imp_enthalpy_vaporization, :out_hvap))
const PROPERTY_EVAL = Dict{String, Function}(
    "vapor_pressure" => imp_saturation_pressure,
    "liquid_density" => imp_saturated_liquid_density,
    "enthalpy_vaporization" => imp_enthalpy_vaporization)

# relative squared error; a failed calculation counts as a 100 % deviation
relative_sq_loss(pred, expt) = isfinite(pred) ? abs2((pred - expt)/expt) : one(pred)

#----- data
tofloat(x::Number) = Float64(x)

```



```

tofloat(x) = (s = strip(string(x)); isempty(s) || lowercase(s) == "nan" ? NaN : parse(Float64, s))
nanmin(v) = (w = filter(!isnan, v); isempty(w) ? NaN : minimum(w))
nanmax(v, default) = (w = filter(!isnan, v); isempty(w) ? default : maximum(w))

function read_rows(path)
    raw, header = readdlm(path, ',', Any, '\n'; header = true, quotes = true)
    names = String.(vec(header))
    return [Dict(names[j] => raw[i, j] for j in eachindex(names)) for i in 1:size(raw, 1)]
end

struct PropertyData
    T::Vector{Float64}
    value::Vector{Float64}
    uncertainty::Vector{Float64}
    npoints::Vector{Int}
end

function select_data(rows, property, Tc)
    sel = filter(rows) do r
        r["property"] == property && string(r["data_type"]) in DATA_TYPES &&
        string(r["acceptance"]) in ACCEPTANCE && tofloat(r["T_K"]) <= TR_MAX*Tc &&
        tofloat(r["value"]) > 0 && !(tofloat(r["uncertainty_percent"]) > MAX_UNCERTAINTY[property])
    end
    T = tofloat.(getindex.(sel, "T_K")); y = tofloat.(getindex.(sel, "value")); u = tofloat.(getindex.(sel,
"uncertainty_percent"))
    if !MERGE_SAME_T
        p = sortperm(T)
        return PropertyData(T[p], y[p], u[p], ones(Int, length(T)))
    end
    groups = Dict{Float64,Vector{Int}}{ }
    for i in eachindex(T)
        push!(get!(groups, round(T[i], digits = 2), Int[]), i)
    end
    Ts = sort!(collect(keys(groups)))
    return PropertyData([mean(T[groups[t]]) for t in Ts],
        [property == "vapor_pressure" ? exp(mean(log.(y[groups[t]]))) : mean(y[groups[t]]) for t in Ts],
        [nanmin(u[groups[t]]) for t in Ts], [length(groups[t]) for t in Ts])
end

function write_estimator_file(dir, property, data::PropertyData)
    method, outcol = METHODS[property]
    w = ones(length(data.T))
    if UNCERTAINTY_WEIGHTS
        umax = nanmax(data.uncertainty, 1.0)
        w = [1/(isnan(ui) ? umax : ui)^2 for ui in data.uncertainty]
        w ./= mean(w)
    end
    path = joinpath(dir, property * ".csv")
    open(path, "w") do io
        println(io, "Clapeyron Estimator")
        println(io, "[method = $method, data_weights_col = weight]")
        println(io, "T,$outcol,weight")
        for i in eachindex(data.T)

```

```

        @printf(io, "%.4f,%.10g,%.6g\n", data.T[i], data.value[i], w[i])
    end
end
return path
end

function dippr_correlation(c, T, Tc)    # eqs 101, 105, 106 -> Pa, mol/m³, J/mol
    A, B, C, D, E = (tofloat(c[k]) for k in ("A", "B", "C", "D", "E"))
    eq = Int(tofloat(c["equation"]))
    eq == 101 && return exp(A + B/T + C*log(T) + D*T^E)
    eq == 105 && return 1e3*A/B^(1 + (1 - T/C)^D)
    eq == 106 && (Tr = T/Tc; return 1e-3*A*(1 - Tr)^(B + C*Tr + D*Tr^2 + E*Tr^3))
    error("DIPPR equation $eq not implemented")
end

#----- fitting
function initial_model(c, order)
    b = PARAMETER_BOUNDS
    return SAFTVRIMP([string(c["species"])]); order,
        userlocations = (Mw = [tofloat(c["Mw_g_per_mol"])], segment = [b.segment[3]], sigma = [b.sigma[3]],
            epsilon = [b.epsilon[3]], lambda_r = [b.lambda_r[3]], lambda_a = [6.0],
            dipole = [tofloat(c["dipole_D"])], quadrupole = [0.0], polarizability = [0.0]))
end

function toestimate()
    b = PARAMETER_BOUNDS
    return [
        Dict{Symbol,Any}(:param => :segment, :lower => b.segment[1], :upper => b.segment[2], :guess =>
b.segment[3]),
        Dict{Symbol,Any}(:param => :sigma, :lower => b.sigma[1], :upper => b.sigma[2], :guess =>
b.sigma[3], :factor => 1e-10),
        Dict{Symbol,Any}(:param => :epsilon, :lower => b.epsilon[1], :upper => b.epsilon[2], :guess =>
b.epsilon[3]),
        Dict{Symbol,Any}(:param => :lambda_r, :lower => b.lambda_r[1], :upper => b.lambda_r[2], :guess =>
b.lambda_r[3]),
    ]
end

function polish(objective, x0, lb, ub)    # Nelder-Mead in x = lb + (ub-lb)·logistic(ξ)
    to_x(ξ) = lb .+ (ub .- lb) ./ (1 .+ exp.(-ξ))
    xc = clamp.(x0, lb .+ 1e-9 .* (ub .- lb), ub .- 1e-9 .* (ub .- lb))
    ξ0 = log.((xc .- lb) ./ (ub .- xc))
    options = Clapeyron.Solvers.OptimizationOptions(maxiter = NELDER_MEAD_ITERATIONS)
    res = Clapeyron.Solvers.optimize(ξ -> objective(to_x(ξ)), ξ0, Clapeyron.Solvers.NelderMead(), options)
    x = to_x(Clapeyron.Solvers.x_sol(res))
    return objective(x) < objective(x0) ? x : x0
end

function fit_compound(c, rows, order)
    key = string(c["key"]); Tc = tofloat(c["Tc_K"])
    dir = mkpath(joinpath(OUT_DIR, "estimator_data", key))
    datasets = Dict{String,PropertyData}()
    files = Tuple{Float64,String}[]

```

```

for property in PROPERTIES
  data = select_data(rows, property, Tc)
  isempty(data.T) && continue
  datasets[property] = data
  push!(files, (PROPERTY_WEIGHTS[property], write_estimator_file(dir, property, data)))
end
estimator, objective, x0, upper, lower = Estimation(initial_model(c, order), toestimate(), files;
                                                    objective_form = relative_sq_loss)

SOLVER_FAILURES[] = 0
t0 = time()
method = ECA(N = ECA_POPULATION, options = Options(iterations = ECA_ITERATIONS, seed = SEED))
x_eca, _ = Metaheuristics.optimize(objective, estimator, method)
x_best = polish(objective, x_eca, lower, upper)
seconds = time() - t0
fitted = return_model(estimator, estimator.model, x_best)
return (key = key, order = order, x = x_best, objective = objective(x_best), model = fitted,
        datasets = datasets, seconds = seconds, failures = SOLVER_FAILURES[])
end

#----- evaluation and output
deviation(calc, expt) = 100*(calc - expt)/expt

function evaluate(result, c, correlations)
  model = result.model
  Tc, Pc, Ttp = tofloat(c["Tc_K"]), tofloat(c["Pc_Pa"]), tofloat(c["Ttp_K"])
  table = Tuple{String,Float64,Float64,Float64,Float64,Int}[]
  stats = Dict{String,Any}()
  for (property, data) in result.datasets
    calc = [PROPERTY_EVAL[property](model, T) for T in data.T]
    dev = deviation.(calc, data.value)
    append!(table, [(property, data.T[i], data.value[i], calc[i], dev[i], data.npoints[i]) for i in
eachindex(data.T)])
    stats[property*_AAD"] = mean(abs, filter(isfinite, dev))
    stats[property*_max"] = maximum(abs, filter(isfinite, dev))
    stats[property*_n"] = length(dev)
    stats[property*_Trange"] = extrema(data.T)
  end
  for corr in filter(r -> string(r["key"]) == result.key, correlations) # extrapolation check
    property = string(corr["property"])
    Ts = range(max(Ttp, 0.45*Tc, tofloat(corr["Tmin_K"])), 0.9*Tc, length = 25)
    dev = [deviation(PROPERTY_EVAL[property](model, T), dippr_correlation(corr, T, Tc)) for T in Ts]
    stats[property*_AAD_vs_DIPPR"] = mean(abs, filter(isfinite, dev))
  end
  Tc_m, Pc_m, Vc_m = crit_pure(model)
  stats["Tc_model"], stats["Pc_model"], stats["Tc_dev"], stats["Pc_dev"] = Tc_m, Pc_m, deviation(Tc_m, Tc),
deviation(Pc_m, Pc)
  return table, stats
end

function write_outputs(results, evaluations, constants, order)
  ok = [(r, e) for (r, e) in zip(results, evaluations) if r.order == order]
  isempty(ok) && return nothing
  params_path = joinpath(OUT_DIR, "SAFTVRIMP_like_fitted_order$(order).csv")

```

```

open(params_path, "w") do io
    println(io, "Clapeyron Database File,,,,,,,,,,,,")
    println(io, "SAFTRIMP Like Parameters (fitted, order = $(order)) [csvtype = like],,,,,,,,,,,,,")
    println(io,
"species,Mw,segment,sigma,epsilon,lambda_r,lambda_a,dipole,quadrupole,polarizability,source,comment")
        for (r, _) in ok
            c = constants[r.key]; m,  $\sigma$ ,  $\epsilon$ ,  $\lambda_r$  = r.x
            @printf(io, "%s,%.5f,%.5f,%.5f,%.4f,%.4f,6.0,%.4f,0.0,0.0,DIPPR 801 (%s),Pvap+rhoL+Hvap
Experimental/Accepted T<=%.2fTc order %d\n",
                c["species"], tofloat(c["Mw_g_per_mol"]), m,  $\sigma$ ,  $\epsilon$ ,  $\lambda_r$ , tofloat(c["dipole_D"]), c["file"],
TR_MAX, order)
        end
    end
    summary_path = joinpath(OUT_DIR, "summary_order$(order).csv")
    props = ("vapor_pressure", "liquid_density", "enthalpy_vaporization")
    open(summary_path, "w") do io
        println(io,
"compound,segment,sigma_A,epsilon_K,lambda_r,dipole_D,objective,n_Psat,AAD_Psat,n_rhoL,AAD_rhoL,n_Hvap,AAD_Hv:
"AAD_Psat_vs_DIPPR,AAD_rhoL_vs_DIPPR,AAD_Hvap_vs_DIPPR,Tc_model_K,Tc_dev,Pc_model_MPa,Pc_dev,seconds")
            for (r, e) in ok
                c = constants[r.key]
                vals = Any[c["species"], r.x..., tofloat(c["dipole_D"]), r.objective]
                for p in props
                    push!(vals, get(e, p*"_n", 0), get(e, p*"_AAD", NaN))
                end
                append!(vals, [get(e, p*"_AAD_vs_DIPPR", NaN) for p in props])
                append!(vals, [e["Tc_model"], e["Tc_dev"], e["Pc_model"]/1e6, e["Pc_dev"], r.seconds])
                println(io, join([v isa AbstractFloat ? @sprintf("%.6g", v) : string(v) for v in vals], ","))
            end
        end
    end
    return params_path, summary_path
end

function print_result(r, e, c)
    m,  $\sigma$ ,  $\epsilon$ ,  $\lambda_r$  = r.x
    @printf(" %-20s order %d  m = %.4f   $\sigma$  = %.4f Å   $\epsilon$ /k = %.2f K   $\lambda_r$  = %.3f  ( $\mu$  = %.3f D)  obj = %.3e  [%0f
s, %d failed evaluations]\n",
        c["species"], r.order, m,  $\sigma$ ,  $\epsilon$ ,  $\lambda_r$ , tofloat(c["dipole_D"]), r.objective, r.seconds, r.failures)
    for (p, label) in (("vapor_pressure", "Psat"), ("liquid_density", "pL"), ("enthalpy_vaporization",
"ΔHvap"))
        haskey(e, p*"_AAD") || continue
        @printf("          %-6s AAD %6.3f %%  max %6.3f %%  (n = %3d, %.1f-%.1f K)  vs DIPPR 0.45-0.9 Tc: %6.3f
%%\n",
            label, e[p*"_AAD"], e[p*"_max"], e[p*"_n"], e[p*"_Trange"]..., get(e, p*"_AAD_vs_DIPPR", NaN))
    end
    @printf("          Tc = %.1f K (%.1f %%)  Pc = %.3f MPa (%.1f %%)\n", e["Tc_model"], e["Tc_dev"],
e["Pc_model"]/1e6, e["Pc_dev"])
    flush(stdout)
end

function main()
    mkpath(OUT_DIR)

```

```

constants = Dict{string{r["key"]} => r for r in read_rows(joinpath(DATA_DIR, "constants.csv"))}
correlations = read_rows(joinpath(DATA_DIR, "correlations.csv"))
keys_to_fit = COMPOUNDS == :all ? sort!(collect(keys(constants))) : COMPOUNDS
results, evaluations = Any[], Any[]
for order in ORDERS, key in keys_to_fit
    c = constants[key]
    r = fit_compound(c, read_rows(joinpath(DATA_DIR, key * "_data.csv")), order)
    table, stats = evaluate(r, c, correlations)
    dir = mkpath(joinpath(OUT_DIR, "deviations_order$(order)"))
    open(joinpath(dir, key * ".csv"), "w") do io
        println(io, "property,T_K,experimental,calculated,deviation_percent,merged_points")
        foreach(row -> @printf(io, "%s,%3f,%8g,%8g,%4f,%d\n", row...), table)
    end
    push!(results, r); push!(evaluations, stats)
    print_result(r, stats, c)
end
for order in ORDERS
    paths = write_outputs(results, evaluations, constants, order)
    paths == nothing || println("wrote ", join(paths, " and "))
end
return results, evaluations
end

```

```

abspath(PROGRAM_FILE) == @__FILE__ && main()

```

Running it

1. Place `fit_saftvrimp.jl`, `extract_dippr.py` and the `SAFTVRIMP/` folder from last time in one directory.
2. Run `pip install xlrd`, then `python3 extract_dippr.py <xls folder> dippr_csv`.
3. Add Clapeyron and Metaheuristics to your Julia environment. I used your local Clapeyron checkout.
4. Run `julia --project fit_saftvrimp.jl`.

On one CPU, each compound took 31–166 s per order, after about a minute of compilation.

A few parts of the listing were not exercised in my run and are untested:

- `write_outputs`, which only runs after every fit finishes;
- the uncertainty-weighting branch;
- the `nanmin`/`nanmax` helpers. They fix a merged-uncertainty bug (`init = NaN` made every value NaN) that only affected the unused weighting option.

I also condensed some formatting and restructured `evaluate`, `write_outputs` and the end of `main` in the listing, so treat those as untested too. The data reading, estimator construction, optimisation and per-compound evaluation are the code that produced the table above, with only cosmetic changes.

9/15/2026, 1:56:05 PM



Continue

9/15/2026, 1:56:17 PM



Resuming the fitting process for the compound data. >

I'll pick up where the run left off: check whether the background fit finished, then test the exact code I pasted (parts of it were restructured and untested), try the effective-dipole fix for the ketones, and deliver the files.

Fixing f-string compatibility before reviewing the Julia listing >

9/17/2026, 10:00:01 AM



Continue

9/15/2026, 7:26:37 PM



Working around a file write conflict by saving separately. >

The old tested version is still on disk; I'll assemble a clean delivery folder instead and write the new script there.

Fixing a Python script's string formatting for compatibility. >

Now the Python portability fix (backslash inside an f-string expression), regenerating the data to confirm identical output:

Working through multiple fixed-dipole and fitted model variants. >

The first variant (order 2, fixed dipole) is written. Checking its output files while the rest runs:

Organizing parameter tables and finalizing summary structure. >

I've tested the listing I gave you, diagnosed the ketone problem, and got most of the fixed-dipole fits done. The full run was still going when I hit the tool limit. The updated script, extracted data and results are in my sandbox, not your outputs folder, and the unfinished fits will be interrupted when this turn ends.

Two bugs in the code I gave you

Everything else ran end to end, including the parts I'd flagged as untested, once these two lines were fixed.

Last line of `fit_saftvrimp.jl` is a parse error. The macro `@__FILE__` swallows `&& main()` as its argument. Replace it with:

```
if abspath(PROGRAM_FILE) == @__FILE__
    main()
end
```

`extract_dippr.py` only works on Python 3.12+. It puts a backslash inside an f-string expression. Replace the `key = ...` line with:

```
number = int(re.findall(r"\d+", os.path.basename(path))[0])
key = f"solvent{number:02d}_{name}"
```

The effective-dipole snippet I suggested was also incomplete. With a fifth parameter, `write_outputs` would shift the summary columns and write the experimental dipole into the parameter file.

Fixed-dipole results

All fits use $\lambda_a = 6$ and the DIPPR experimental dipole. AADs are in percent. Tetrahydrothiophene at third order did not finish, and for cyclohexanone at third order I have the parameters but not the deviations.

Solvent	Order	m	σ (Å)	ϵ/k (K)	λ_r	P_sat / p_L / ΔH_{vap} AAD (%)	T_c	P_c	Bounds hit
tetrahydrofuran	2	1.6210	4.1608	370.03	14.083	0.48 / 0.23 / 0.96	+6.2 %	+40 %	none
tetrahydrofuran	3	1.5708	4.2123	385.03	14.715	0.46 / 0.22 / 0.89	+3.1 %	+20 %	none
tetrahydropyran	2	2.0140	4.0942	325.24	12.213	0.51 / 0.08 / 0.20	+3.8 %	+22 %	none
tetrahydropyran	3	2.0005	4.1044	328.12	12.320	0.51 / 0.08 / 0.20	+1.6 %	+7 %	none
tetrahydrothiophene	2	1.7841	4.1840	362.56	11.409	0.97 / 0.05 / 0.54	+5.9 %	+38 %	none
2-butanone	2	1.1035	5.000	372.61	11.874	1.64 / 0.48 / 2.03	+11.6 %	+63 %	σ
2-butanone	3	1.1035	5.000	372.13	11.844	1.60 / 0.48 / 1.99	+8.3 %	+40 %	σ
cyclohexanone	2	1.3797	5.000	309.42	8.000	2.80 / 0.92 / 2.60	+12.2 %	+50 %	σ, λ_r
cyclopentanone	2	1.2905	5.000	272.43	8.000	6.32 / 5.72 / 12.0	+7.0 %	+32 %	σ, λ_r
cyclopentanone	3	1.2907	5.000	272.43	8.000	6.31 / 5.74 / 12.0	+5.8 %	+24 %	σ, λ_r



Use the tetrahydrofuran, tetrahydropyran and tetrahydrothiophene parameters; order 3 is preferable where available because it halves the T_c overshoot. The ketone rows are not usable.

Why the ketones fail

The IMP replaces the orientational free energy $-kT \ln(1+x)$ with its linearization $-kT \cdot x$, where $x = \hat{\mu}^4 / (3T^2 \sigma^6)$ at contact ($\hat{\mu}$ in $K \cdot \text{\AA}^3$, σ in $\text{\AA}$). I added x at the normal boiling point to the script's output.

- **Ethers and tetrahydrothiophene:** x is 0.04–0.09, where the linearization is accurate to a few percent.
- **Cyclopentanone:** with realistic geometry, x is 0.64 at its boiling point and 1.4 at 273 K. The linear form then overstates polar attraction by 30–60 %, worst at low temperature, which is the trend in the deviations.

The fits try to compensate by inflating σ to dilute the polar term. Two diagnostic runs on cyclopentanone (order 3) confirm this:

- **Widened bounds ($\sigma \leq 6.5 \text{ \AA}$, $\lambda r \geq 6.5$):** the model collapses to a single sphere ($m = 1.00$, $\sigma = 5.34 \text{ \AA}$) and is still poor (4.8 / 1.1 / 5.1 %).
- **Dipole fitted within 0 to its experimental value:** the fit becomes good (1.02 / 0.20 / 0.81 %, no bounds hit), but μ_{eff} falls to 0.33 D, a tenth of the experimental 3.24 D.

So for the ketones an effective dipole gives good pure-component accuracy only by nearly switching the polar term off. The model then behaves like non-polar SAFT-VR Mie and loses the polar contribution you would want for mixtures. A real fix needs a polar term that saturates at low temperature, which means changing the model rather than the fitting. I only ran the effective-dipole fit for cyclopentanone; 2-butanone and cyclohexanone are probably similar but unverified.

Not finished

- The effective-dipole fits for the other five solvents, at both orders.
- Tetrahydrothiophene at order 3.
- The plot of deviations.
- Copying everything to your outputs folder.

The updated script in my sandbox already does all of this. It selects the variant from the command line (`--dipole=fixed,fitted`, `--orders=2,3`, custom bounds), flags bound-limited parameters, and reports the validity number x . If you ask me to continue, I'll rerun those fits and deliver the script, data, results and figure as files.

9/15/2026, 7:59:05 PM

[Continue](#)

9/15/2026, 8:04:07 PM



Sequencing remaining fits and setting up automatic result summaries. >

I'll finish the work this turn. The previous background run was likely killed again, so first I'll check what survived.

Running curve fits and checking the delivery script's file handling. >

The summary generator works and the effective-dipole fits are running, with automatic re-delivery after each order. Now the README:

Summarizing parameter fits, key findings, and data caveats. >

All 24 fits (six solvents, orders 2 and 3, experimental and effective dipole) finished and are in your outputs folder. The folder contains the fitting script, the extracted data, per-variant parameter files, a results summary and a deviation plot.

The delivered script replaces the listing I pasted earlier. It fixes both bugs in that listing (the Julia parse error and the Python 3.12-only f-string). It also adds command-line options for the variant (`--dipole=fixed,fitted`, `--orders`, custom bounds), flags parameters that end on a search bound, and reports x , a measure of whether the IMP approximation holds (below). The README gives the commands.

Recommended parameters (order 3, $\lambda a = 6$)

Deviations are average absolute deviations (%) from the fitted experimental data.

Solvent	Dipole	m	σ (Å)	ϵ/k (K)	λr	μ (D)	$P_{\text{sat}} / \rho_L / \Delta H_{\text{vap}}$ (%)	T_c dev.
tetrahydrofuran	experimental	1.5708	4.2123	385.03	14.715	1.631	0.46 / 0.22 / 0.89	+3.1 %
tetrahydropyran	experimental	2.0005	4.1044	328.12	12.320	1.550	0.51 / 0.08 / 0.20	+1.6 %
tetrahydrothiophene	experimental	1.7570	4.2071	370.68	11.656	1.901	0.97 / 0.05 / 0.53	+3.6 %
2-butanone	effective	1.9065	4.0864	409.99	18.260	0.002 (exp. 2.779)	0.50 / 0.40 / 1.42	+3.9 %
cyclohexanone	effective	1.6553	4.6218	552.47	19.929	0.319 (exp. 3.088)	0.60 / 0.19 / 0.82	+5.1 %
cyclopentanone	effective	1.7935	4.2290	488.16	18.213	0.333 (exp. 3.238)	1.02 / 0.20 / 0.81	+4.1 %

Critical pressures come out 7–40 % too high in all six sets. The fits exclude near-critical data, so this is normal for SAFT.

What the dipole comparison showed

Ketones: the data reject the experimental dipole. With the experimental value fixed, the fits end on their bounds and miss by several percent; cyclopentanone is off by 6 % in vapour pressure and 12 % in ΔH_{vap} . Freeing the dipole improves the objective 4× for 2-butanone, 11× for cyclohexanone and 58× for cyclopentanone, but only by shrinking it to 0–0.33 D. At that point the polar term is almost off and the model behaves like non-polar SAFT-VR Mie. These parameters describe the pure ketones well, but they won't supply the polar contribution SAFT-VR IMP is meant to add in mixtures.

The cause is the model's approximation. x is the strength of the polar interaction at contact, evaluated at the normal boiling point. It is 0.3–0.6 for the ketones, well outside the range where the IMP's linearized polar term is accurate. Fixing that needs a polar term that saturates at low temperature, which is a change to the model rather than to the fit.

Ethers and sulfide: the dipole can't be determined from these data. Their x is only 0.04–0.09, so the IMP is valid. Freeing the dipole improves the objective by at most 13 %. The fitted value scatters without physical meaning: tetrahydrothiophene gets 0.01 D at order 2 and 0.97 D at order 3, while ϵ and σ trade off against it. For these three, keep the experimental dipole: the accuracy is the same and the value is physically grounded.

Limitations of the data

Accepted liquid densities only reach about 0.6 T_c for every solvent. Tetrahydropyran's vapour pressures cover only 0.52–0.63 T_c . The results summary includes a comparison against DIPPR's correlations up to 0.9 T_c , so you can see how the parameters

extrapolate beyond the measured range.

The script also works around a bug in your Clapeyron master (`x0_sat_pure_crit` called without its `T, crit` arguments in `initial_guess.jl`). The README describes the one-line upstream fix.

	README Md	
	RESULTS Md	
	fit_saftvrimp Jl	
	extract_dippr Py	
	deviations_overview Png	
	SAFTVRIMP_like Csv	
	SAFTVRIMP_like Csv	
	SAFTVRIMP_fit Zip	

9/15/2026, 8:23:33 PM



How do the results of SAFT-VR IMP compare with SAFT-gamma Mie?

9/15/2026, 8:29:51 PM



Planning a comparison script for deviations and criticals. >

I'll check which of the six solvents Clapeyron's SAFT- γ Mie group database can represent, then evaluate it against the same experimental data points the IMP fits used.

Ran command >

9/15/2026, 8:30:30 PM



Continue

9/16/2026, 9:28:12 AM



Tabulating accuracy and extrapolation results across compounds. >

SAFT-γ Mie cannot represent tetrahydrothiophene, because Clapeyron's database has no sulfur group. For the other five, the fitted SAFT-VR IMP parameters describe the pure-fluid data far better, but they only have that edge because they were fitted to these data. SAFT-γ Mie, predicting purely from its group parameters, still gets critical temperatures closer and is competitive for 2-butanone.

I evaluated SAFT-γ Mie on exactly the data points used for the fits (Experimental/Accepted, $T \leq 0.9 T_c$) and compared it with the order-3 IMP parameter sets.

Solvent	Model	P _{sat} AAD	ρ _L AAD	ΔH _{vap} AAD	T _c dev.
tetrahydrofuran	SAFT-γ Mie	3.1 %	19.8 %	0.8 %	+1.4 %
	IMP, experimental μ	0.46 %	0.22 %	0.89 %	+3.1 %
	IMP, effective μ	0.44 %	0.22 %	0.87 %	+2.9 %
tetrahydropyran	SAFT-γ Mie	21.9 %	17.7 %	5.2 %	+1.4 %
	IMP, experimental μ	0.51 %	0.08 %	0.20 %	+1.6 %
	IMP, effective μ	0.49 %	0.06 %	0.09 %	+0.8 %
2-butanone	SAFT-γ Mie	3.2 %	0.42 %	1.0 %	+2.8 %
	IMP, experimental μ	1.6 %	0.48 %	2.0 %	+8.3 %
	IMP, effective μ	0.50 %	0.40 %	1.4 %	+3.9 %
cyclohexanone	SAFT-γ Mie	0.71 %	15.5 %	1.2 %	+1.4 %
	IMP, experimental μ	2.8 %	0.92 %	2.6 %	+10.9 %
	IMP, effective μ	0.60 %	0.19 %	0.82 %	+5.1 %
cyclopentanone	SAFT-γ Mie	86.6 %	8.1 %	31.3 %	+28.1 %
	IMP, experimental μ	6.3 %	5.7 %	12.0 %	+5.8 %
	IMP, effective μ	1.0 %	0.20 %	0.81 %	+4.1 %

Where SAFT-γ Mie does well.

- **2-butanone:** it matches or beats the fitted experimental-dipole IMP set on everything except vapour pressure, and gives the best T_c of all three.
- **Critical temperatures:** it lands within 1.4–2.8 % for four of the five solvents, closer than most IMP sets. That is partly because the IMP fits deliberately excluded near-critical data.
- **Vapour pressure:** it is good for THF and cyclohexanone.

Where it fails. The ring molecules containing the cyclic-ether (**cy0**) or cyclic-ketone (**cc=0**) groups come out 16–21 % too low in liquid density.

- **Cause:** the ring CH₂ group itself is fine. Cyclohexane built from it gives P_{sat} = 13.02 kPa and ρ_L = 9.16 mol/L at 298.15 K, essentially the experimental values. The problem is the **cy0** and **cc=0** parameters, which have no source recorded in the database and don't transfer to these molecules.



- **Cyclopentanone:** its failure comes from a dedicated five-membered-ring interaction between cCH_2 and $cC=O$ ($\epsilon = 927$ K), which puts T_c at 800 K instead of 624.5 K. Building it with ordinary ring CH_2 instead improves vapour pressure to 9 % and T_c to 648 K, but the density is still 16 % low.

How to read the comparison. The IMP sets were regressed on these very data, so their low deviations are expected rather than a measure of predictive power. The effective-dipole ketone sets are, in practice, fitted SAFT-VR Mie models with the polar term nearly switched off. For these pure solvents, use the fitted IMP parameters. Treat SAFT- γ Mie as a reliable predictive alternative here only for 2-butanone, and don't rely on its **cy0** or **cC=0** groups for densities.

The comparison script, results table, deviation plot and updated README (with a new comparison section) are below.

	summary Csv	
	deviations_overview Png	
	compare_saftgammamie Jl	
	README Md	
	SAFTVRIMP_fit Zip	

9/16/2026, 9:39:36 AM



Listing gaps, bugs, and comparisons still unresolved in the analysis. >

Stage 1: Inventory, gaps, and outline

(a) Inventory of facts from the conversation

Tools, versions, and provenance

- LLM: Claude Fable 5.1 (Anthropic), used through the claude.ai chat interface with a Linux sandbox (bash, file tools). One CPU, Julia downloaded during the session.
- Software: Julia 1.11.7; Clapeyron.jl master (v0.6.28, uploaded as a zip by the researcher); ForwardDiff.jl; Metaheuristics.jl 3.5.0; Python 3 with xldr for the legacy `.xls` files.
- Source papers uploaded by the researcher: Müller & Gelb, *Ind. Eng. Chem. Res.* 42, 4123 (2003) (IMP, TQMD); Jervell, Maltby, Aasen, Hammer, Wilhelmsen, *J. Chem. Phys.* 164, 114101 (2026) (SAFT-VR Sum); Walker, Yew, Riedemann, *Ind. Eng. Chem. Res.* 61, 7130 (2022) (Clapeyron.jl).
- Standard references used inside the model: Lafitte et al., *J. Chem. Phys.* 139, 154504 (2013); Gil-Villegas et al. 1997; Barker-Henderson; Boublik 1986; Percus-Yevick; Aasen et al. 2019 (SAFT-VRQ Mie, source of the d(T) quadrature strategy).

The researcher's brief (turn 1)

- Formulate and implement a SAFT-VR variant using the IMP potential as a sum of Sutherland potentials in Clapeyron.jl, referencing SAFT-VR Mie and SAFT-VR Sum.
- Requested potential: $\phi = \infty$ for $r < \sigma$, $-\sum_k \epsilon_k (\sigma/r)^{\lambda_k}$ for $r \geq \sigma$; second-order Barker-Henderson expansion $a^{\text{mono}} = a^{\text{hs}} + \beta a_1 + \beta^2 a_2$; T-dependent d(T) with ForwardDiff compatibility; chain term with ρ_m and $x_{\{s,i\}}$; struct names `SAFTVRIMPParam`, `SAFTVRIMP <: SAFTModel`; functions `a_mono`, `a_chain`, `a_res`; validation with n-hexane.

Model as delivered (SAFTVRIMP.jl)

- IMP definition: orientationally averaged multipolar potential of Müller & Gelb; leading cumulant $\Gamma \approx -(\beta/2)\langle u^2 \rangle_\omega$; $\Gamma^{\mu\mu} = -\beta \mu^2 k^2 \mu_L^2 / (3r^6)$, $\Gamma^{\mu Q} = -\beta (\mu^2 k^2 Q_L^2 + Q^2 k^2 \mu_L^2) / (2r^8)$, $\Gamma^{QQ} = -7\beta Q^2 k^2 Q_L^2 / (5r^{10})$, $\Gamma^{\mu\alpha} = -(\mu^2 k^2 \alpha_L + \alpha^2 k^2 \mu_L^2) / r^6$.
- Pair potential: $u_{kl}(r; T) = \sum_{n=1}^5 c_{\{kl,n\}}(T) (\sigma_{kl}/r)^{\lambda_n}$, with (λ, c) : $(\lambda_r, +C\epsilon)$, $(\lambda_a, -C\epsilon)$, $(6, -[\hat{\mu}_k^2 \hat{\alpha}_l + \hat{\alpha}_k \hat{\mu}_l^2 + \hat{\mu}_k^2 \hat{\mu}_l^2 / (3T)] / \sigma^6)$, $(8, -(\hat{\mu}_k^2 \hat{Q}_l^2 + \hat{Q}_k^2 \hat{\mu}_l^2) / (2T\sigma^8))$, $(10, -7\hat{Q}_k^2 \hat{Q}_l^2 / (5T\sigma^{10}))$; C the Mie prefactor. Brief's $\epsilon_k = -C_n$.
- Segment moments: $\hat{\mu}^2 = \mu^2 / (4\pi\epsilon_0 m)$, $\hat{Q}^2 = Q^2 / (4\pi\epsilon_0 m)$, $\hat{\alpha} = \alpha / m$; conversion $(1 \text{ D})^2 / (4\pi\epsilon_0 k_B) = 7242.97 \text{ K} \cdot \text{\AA}^3$.
- Well-posedness checks at construction: $\lambda_r > \text{largest active attractive exponent}$; $\lambda_a > 4$.
- Effective quantities $\sigma_{\text{eff}}(u=0)$, r_{min} , $\epsilon_{\text{eff}} = -u(r_{\text{min}})$, $\alpha = -\epsilon_{\text{eff}}^{-1} \sum c_n x_{\text{eff}}^{-\lambda_n} / (\lambda_n - 3)$; solved by Newton on $F(s) = \ln w_1 - \lambda_1 s - \ln \sum_{n \geq 2} w_n e^{-\lambda_n s}$ (strictly decreasing, concave; global convergence); three extra Newton steps at the converged root so nested ForwardDiff duals are exact; closed-form Mie branch when no multipole term is active. Closed-form Keesom mapping: $F = 1 + |c| / (C\epsilon)$, $\sigma_m = \sigma F^{-1} \{-1 / (\lambda_r - \lambda_a)\}$, $\epsilon_m = \epsilon F^{\lambda_r / (\lambda_r - \lambda_a)}$; LJ case $\epsilon_m = \epsilon F^2$, $\sigma_m = \sigma F^{-1/6}$, $F = 1 + \mu^4 / (12T)$.
- Barker-Henderson $d(T) = \int_0^\infty \{\sigma_{\text{eff}}\} [1 - e^{-\beta u}] dr$ via 10-point Gauss-Laguerre ($\theta = c_1/T > 1$) or Gauss-Legendre above the machine-precision cutoff; additive d_{kl} .
- BMCSL hard-sphere term; ζ_x over d_{kl} , $\tilde{\zeta}_x$ over $\sigma_{\text{eff},kl}$.
- $a_1 = -2\pi \rho_s d^3 \sum c_n x_0^{\lambda_n} \Phi(\lambda_n)$, $\Phi = a_1^{\text{HS}} + B$; $a_2 = \pi \rho_s K^{\text{HS}} (1 + \chi) d^3 \sum_{\{n,m\}} c_n c_m x_0^{\lambda_n + \lambda_m} \Phi(\lambda_n + \lambda_m)$ (15 unordered pairs); optional a_3 (Lafitte correlation with ϵ_{eff} , α); chain: $\ln y(\sigma_{\text{eff}}) = \ln g^{\text{HS}_d}(x_{\text{eff}}) + (\tau g_1 + \tau^2 g_2) / g^{\text{HS}}$, $g_1 = [\sum \lambda_n c_n x_0^{\lambda_n} \Phi - 3\partial(\rho_s S_1) / \partial \rho_s] / \epsilon_{\text{eff}}$, $g_2 = (1 + \gamma_c) g_2^{\text{MCA}}$.
- Mixing: Lorentz-Berthelot σ , Hudson-McCoubrey ($\sqrt{\epsilon}$) with optional k_{ij} , λ rule; multipolar cross coefficients need no combining rule (this is why the struct stores μ , Q , α rather than ϵ_k).



- Design departures from the brief, with stated reasons: soft Mie core instead of hard core (hard core gives $d = \sigma$, no T dependence; chain term does not commute with the $\lambda r \rightarrow \infty$ limit); contact at σ_{eff} so $y = g$; default `order = 2`, `order = 3` adds a_3 and reproduces SAFTVRMie exactly at zero moments.
- Sign inconsistency identified in SAFT-VR Sum paper: tail term of g_1 has minus sign in their eq. (29) and plus sign in eq. (32); only the minus sign reproduces SAFT-VR Mie; factor $\epsilon/\epsilon_{\text{eff}}$ needed when c_n written as ϵC_n .
- IMP validity measure $x = \hat{\mu}^4/(3T^2\sigma^6)$ at contact (linearisation of $-kT \ln(1+x)$).
- Options struct `SAFTVRIMPOptions(order)` (non-splittable), `default_locations`, `transform_params`, `recombine_impl!`, `sutherland_terms`, `imp_effective_parameters`.
- Cost: hexane pressure 1.7 μs (SAFTVRMie 1.3 μs); benzene (Q active) 3.1 μs ; DCE+cyclohexane mixture 14.2 μs (SAFTVRMie hexane+decane 4.0 μs); benzene C_v 5.5 μs .
- Deliverables: `SAFTVRIMP.jl`, `database/SAFTVRIMP_like.csv` (hexane, decane with SAFT-VR Mie parameters; five Müller-Gelb fluids with Q/μ converted to $D\cdot\text{\AA}/D$: benzene $Q = 12.2615$, CO_2 4.0982, naphthalene 13.4997 (not in CSV), DCE $\mu = 2.0899$, sulfolane 13.7005), test script, derivation document, in-tree patch (applied cleanly, precompiles), README.

Verification of the model (97 tests, all pass, ~3.5 min)

- Zero moments + order 3 $\equiv$ `SAFTVRMie`: a_{res} , p , μ_{res} agree $< 10^{-13}$ (relative) for hexane and hexane+decane at liquid, vapour, supercritical states including 1200 K; normal boiling point 341.86441622 K identical; $a_{\text{mono}} + a_{\text{chain}} = a_{\text{res}}$.
- Dipolar LJ $\mu^* = 2$ vs analytic Keesom mapping: σ_{eff} , ϵ_{eff} , d , $a_{\text{res}} \leq 5 \times 10^{-15}$ between 120–3000 K; first and second T-derivatives of ϵ_{eff} match analytic to 13 digits.
- $\lambda = 6, 8, 10$ slots, chains ($m = 1.5, 2$), unlike pairs vs mapped `SAFTVRMie`: $\leq 7 \times 10^{-14}$.
- AD vs Richardson finite differences (polar chain + decane, all five terms active, order 2): $\partial p/\partial T$, $\partial p/\partial V$, $\partial p/\partial z_1$, $\partial^2 A^{\text{res}}/\partial T^2 \leq 4 \times 10^{-11}$; C_v vs $\partial U/\partial T$ 6×10^{-12} ; Euler relation; dx_{eff}/dT and $d\epsilon_{\text{eff}}/dT$ vs IFT/envelope theorem to 13 digits.
- Clapeyron equation for quadrupolar benzene at 450 K satisfied to 4×10^{-11} .

Predictions with Müller-Gelb parameters (not refitted)

- T_c (order 2 / order 3 / order 2 moments off / exp): benzene 571.5/553.5/488.2/562.0 K; CO_2 341.7/331.0/292.6/304.1; 1,2-dichloroethane 599.2/578.9/568.6/561.6; cyclohexane 560.6/540.6/560.6/553.6; n-octane 588.2/567.2/588.2/568.7.
- LJ $T_c^* = 1.3122$ (o3), 1.3608 (o2); $\mu^2 = 2$: 1.8329 (o3), 1.8847 (o2); ρ_c 0.3003/0.3105 and 0.3264/0.3369.
- n-hexane: order 2 $T_b = 341.88$ K, $T_c = 530.4$ K; order 3 $T_c = 515.1$ K; exp ≈ 341.9 K, 507.8 K, 3.03 MPa.
- DCE + cyclohexane: azeotrope predicted at 1 atm; pure boiling points ≈ 329 K and 310 K (25–45 K too low).

Parameter estimation (turn 2-3)

- Data: six DIPPR 801 `.xls` exports; compounds and DIPPR dipoles: tetrahydrofuran 1.631 D, tetrahydropyran 1.550, 2-butanone 2.779, cyclohexanone 3.088, cyclopentanone 3.238, tetrahydrothiophene 1.901. Extraction by Python (`extract_dippr.py`), block-structured raw-data sheet, DIPPR eqs. 101/105/106 exported.
- Selection: P_{sat} , ρ_L , ΔH_{vap} ; Experimental + Accepted; $T \leq 0.9T_c$; uncertainty $\leq 5/3/5$ %; repeated temperatures merged (geometric mean for P_{sat}). Point counts noted: 2-butanone 136/46/17 merged points; cyclopentanone 62/15/2; THF 62/12/5; THP 6 ρ_L , 4 ΔH_{vap} (P_{sat} 0.52–0.63 T_c only); accepted densities reach $\approx 0.6 T_c$ for all.
- Fitted: m , σ , ϵ , λr ($\lambda a = 6$, $Q = 0$, $\alpha = 0$; μ fixed at DIPPR or fitted in $[0, \mu_{\text{exp}}]$); bounds m 1–4, σ 3–5 $\text{\AA}$, ϵ 150–700 K, λr 8–30. Objective: Clapeyron relative MSE per property, weights 1 : 1 : 0.5; NaN $\rightarrow$ 100 % penalty; ECA ($N = 40$, 150 iterations, seed 20260915) + Nelder–Mead (400 it.) in logistic-transformed variables. 31–166 s per compound per variant.
- Clapeyron bug found: `x0_sat_pure(model, T, crit)` calls `x0_sat_pure_crit(satmodel)` without `T, crit` (MethodError in saturation retries above 0.6 T_c); overridden for SAFTVRIMP models.
- Results, fixed dipole, order 3 (m , $\sigma/\text{\AA}$, ϵ/K , λr ; AAD $P_{\text{sat}}/\rho_L/\Delta H_{\text{vap}}$ %; T_c dev): THF 1.5708, 4.2123, 385.03, 14.715; 0.46/0.22/0.89; +3.1 %. THP 2.0005, 4.1044, 328.12, 12.320; 0.51/0.08/0.20; +1.6 %. THT 1.7570, 4.2071, 370.68, 11.656; 0.97/0.05/0.53; +3.6 %. 2-butanone 1.1035, 5.000*, 372.13, 11.844; 1.60/0.48/1.99; +8.3 %. Cyclohexanone: 2.79/0.92/2.61,

+10.9 % (from comparison run; parameters not printed). Cyclopentanone 1.2907, 5.000*, 272.43, 8.000*; 6.31/5.74/12.0; +5.8 %.

- Fixed dipole, order 2: THF 1.6210, 4.1608, 370.03, 14.083; 0.48/0.23/0.96; +6.2 %. THP 2.0140, 4.0942, 325.24, 12.213; 0.51/0.08/0.20; +3.8 %. THT 1.7841, 4.1840, 362.56, 11.409; 0.97/0.05/0.54; +5.9 %. 2-butanone 1.1035, 5.000*, 372.61, 11.874; 1.64/0.48/2.03; +11.6 %. Cyclohexanone 1.3797, 5.000*, 309.42, 8.000*; 2.80/0.92/2.60; +12.2 %. Cyclopentanone 1.2905, 5.000*, 272.43, 8.000*; 6.32/5.72/12.0; +7.0 %.
- Effective dipole, order 3: THF 1.6048, 4.1793, 385.56, 14.915, μ 1.548; 0.44/0.22/0.87; +2.9 %. THP 2.2219, 3.9434, 321.28, 12.594, μ 0.000; 0.49/0.06/0.09; +0.8 %. 2-butanone 1.9065, 4.0864, 409.99, 18.260, μ 0.002; 0.50/0.40/1.42; +3.9 %. Cyclohexanone 1.6553, 4.6218, 552.47, 19.929, μ 0.319; 0.60/0.19/0.82; +5.1 %. Cyclopentanone 1.7935, 4.2290, 488.16, 18.213, μ 0.333; 1.02/0.20/0.81; +4.1 %. THT 1.4255, 4.5895, 552.51, 19.814, μ 0.972; 0.93/0.06/0.68; +3.1 %.
- Effective dipole, order 2: only THF printed (1.5893, 4.2154, 434.21, 18.214, μ 0.850) and THT $\mu \approx 0.01$ D noted.
- Objective improvement fixed $\rightarrow$ fitted dipole: $\times 4$ (2-butanone), $\times 11$ (cyclohexanone), $\times 58$ (cyclopentanone); ≤ 13 % for ethers/sulfide. Keesom x at T_b : 0.04–0.09 (ethers, sulfide), 0.3–0.6 (ketones); cyclopentanone 0.64 at T_b , 1.4 at 273 K. P_c overpredicted 7–63 % (fixed), 7–40 % (recommended sets).
- Diagnostic: cyclopentanone with widened bounds ($\sigma \leq 6.5$ Å, $\lambda r \geq 6.5$) $\rightarrow m = 1.00$, $\sigma = 5.34$ Å, 4.8/1.1/5.1 %. 2-butanone fixed-dipole deviation trend: $Psat$ 6.7 % low at 273 K, 8.9 % low at 483 K, ≈ 1 % high in between; ΔH_{vap} 2–3.5 % high.
- Physical rationale given for effective dipole: IMP keeps only the leading $1/T$ cumulant, exact form $-kT \ln(1+x)$ is smaller in magnitude; μ_{eff} compensates; recommendation from Müller & Gelb to treat μ as adjustable for strongly polar fluids. (Condensed-phase polarisation and charge-distribution simplification are *not* discussed in the conversation.)

SAFT- γ Mie comparison (computed in the conversation, Clapeyron group database)

- Groups: THF 4 cCH2_5 + cyO; THP 5 cCH2 + cyO; 2-butanone CH3 + CH2 + CH3CO; cyclohexanone 5 cCH2 + cC=O; cyclopentanone 4 cCH2_5 + cC=O; THT not representable (no S group).
- AAD $Psat/pL/\Delta H_{vap}$, T_c dev, P_c dev: THF 3.08/19.79/0.81, +1.4, -10.3 ; THP 21.92/17.70/5.19, +1.4, -14.0 ; 2-butanone 3.15/0.42/1.04, +2.8, +22.2; cyclohexanone 0.71/15.53/1.17, +1.4, +3.4; cyclopentanone 86.56/8.06/31.27, +28.1, +44.5. Biases and vs-DIPPR (0.45–0.9 T_c) columns also available; IMP sets in same table.
- Cyclohexane from 6 cCH2: $Psat$ 13.02 kPa, pL 9.16 mol/L at 298.15 K (ring CH₂ group sound). Cyclopentanone with 4 cCH2 instead: 9.09/16.21/2.84 %, T_c 648.0 K (exp 624.5 K); cCH2_5–cC=O unlike $\epsilon = 926.96$ K drives T_c to 800.2 K. cyO and cC=O parameters carry no source in the database.

Errors and issues that arose

- LLM: an aborted file-creation tool call; Newton tolerance 4-eps too strict (relaxed to 10^{-12}); Julia parse error in pasted listing (`@_FILE && main()`); Python f-string backslash (needs 3.12+); `@elapsed` scoping; `nanmin(init=NaN)` bug in untested weighting branch; DIPPR raw-data parser initially mis-assigned acceptance codes (metadata rows share rows with the first data points) and mistook "Notes:" rows for headers; initial effective-dipole snippet incomplete (would shift summary columns); parts of the pasted listing explicitly flagged as restructured/untested, later tested and corrected.
- Environment: background jobs killed between turns; Julia had to be installed; token/tool limits interrupted delivery twice.
- Caught by: the LLM's own tests (97-test suite, smoke tests, verbatim re-execution of the pasted listing), not by the researcher within the conversation.

(b) Missing information (will be marked as placeholders)

1. SAFT-VR Mie *without* a polar term fitted to the six solvents (never run; only the $\mu_{eff} \approx 0$ sets for THP and 2-butanone approximate it, and "moments off" T_c values exist only for the Müller-Gelb fluids).
2. Full order-2 effective-dipole table (only THF printed).
3. Order-3 fixed-dipole cyclohexanone parameters (deviations known, parameter values not shown).
4. P_c deviations per compound for the fixed-dipole sets (only ranges quoted) and experimental T_c/P_c values except cyclopentanone (624.5 K).
5. Any literature SAFT- γ Mie results for these compounds; sources of the cyO/cC=O group parameters.

6. Statistical uncertainties on parameters/AAD; independent human audit of the derivation; molecular-simulation benchmarks; mixture VLE tests with fitted parameters; association term.
7. Physical arguments about condensed-phase polarisation for the effective dipole (not in the conversation; will be flagged as my addition or [CITATION NEEDED]).
8. Exact number of raw vs merged data points for THP, THT, cyclohexanone.
9. Wall-clock time of the whole human-LLM session; tokens used.

(c) Section-by-section outline

1. **Title / Abstract** — "LLM-assisted development of an equation of state: a case study of SAFT-VR IMP for polar fluids" (working title); ≤ 250 words.
2. **Introduction** — question posed; case study (one researcher, one LLM, three sessions: model, fitting, comparison); contributions (new EoS + code, 97 verification tests, six-solvent parameterisation, comparison with SAFT- γ Mie, documented error record).
3. **Background: SAFT equations of state** — $A = A^{\text{ideal}} + A^{\text{mono}} + A^{\text{chain}} + A^{\text{assoc}}$; Wertheim TPT1; SAFT-VR Mie and SAFT- γ Mie; uses (associating fluids, polymers, multicomponent mixtures, carbon capture/solvent design [CITATION NEEDED where needed]); top-down SAFT- γ force fields and why the mapping works.
4. **Why a new SAFT EoS is hard** — six difficulties as requested, each illustrated with what actually arose (kernels Φ , B-integrals; $d(T)$ quadrature; Clapeyron-equation test; saturation-solver bug; σ - μ_{eff} degeneracy; six compounds only).
5. **Polar molecules and the IMP** — implicit treatment limitations; polar SAFT landscape (Gross-Vrabec, Jog-Chapman, SAFT-VR Mie GV [VERIFY]); IMP equations, Sutherland-sum form, effective quantities, validity parameter x .
6. **Methods** — 6.1 LLM and workflow; 6.2 implementation (Julia, Clapeyron conventions, ForwardDiff, fused disp/chain pass); 6.3 verification suite (definition of "working"); 6.4 parameter estimation; 6.5 the effective-dipole suggestion and trade-offs; 6.6 error record.
7. **Results** — Table 1 verification; Table 2 M&G predictions; Table 3 fitted parameters (order 3 fixed/effective); Table 4 AADs for SAFT- γ Mie vs IMP variants; Figure placeholders (deviation overview; comparison overview); fairness discussion (fitted vs transferable, parameter count, data range, T_c).
8. **Limitations of LLMs in scientific work** — audit burden; plausible-but-wrong risk; observed limitations (tool/context interruptions, untested listings, self-reported honesty); recommended verification practices.
9. **Conclusion** — evidence-based claim, oversight caveat, future work (refit with μ fixed and saturating polar term, association, mixtures, simulation benchmarks, independent derivation audit).
10. **Back matter** — AI-use statement; data/code availability; references; Appendix A derivation (§1–7 of the derivation document); Appendix B code excerpts (`imp_sutherland`, `imp_effective`, `imp_dispchain` core loop, fitting objective).

Please confirm, or adjust the outline, and I will write the paper.

9/17/2026, 9:19:21 AM





confirm

9/17/2026, 9:21:08 AM



Drafting a lengthy LaTeX paper on the requested topic. >

```
\documentclass[11pt]{article}
\usepackage[a4paper,margin=2.5cm]{geometry}
\usepackage{amsmath,amssymb,booktabs,graphicx,hyperref,listings,xcolor}
\usepackage[numbers,sort&compress]{natbib}
\lstset{basicstyle=\ttfamily\footnotesize,breaklines=true,columns=fullflexible}
\newcommand{\todo}[1]{\textcolor{red}{[#1]}}

\title{Large language models as collaborators in molecular thermodynamics:\\
a case study of the LLM-assisted development of SAFT-VR IMP,\\
an equation of state for polar fluids}
\author{\todo{AUTHORS}}
\date{}

\begin{document}
\maketitle

\begin{abstract}
Whether large language models (LLMs) can contribute to non-trivial science, beyond summarising literature and writing routine code, is an open question. We report a case study in which an LLM (Claude Fable-5.1, Anthropic) was asked to formulate, implement, verify and parameterise a new equation of state of the statistical associating fluid theory (SAFT) family. The target was a SAFT-VR variant for polar and dipolar molecules in which the segments interact through the isotropic multipolar potential (IMP) of Muller and Gelb, expressed as a temperature-dependent sum of Sutherland terms and treated with the Barker-Henderson perturbation theory of SAFT-VR Mie and SAFT-VR Sum. Over three sessions the LLM produced a step-by-step derivation, a 600-line implementation in the open-source Julia package Clapeyron.jl, a suite of 97 verification tests, a parameter-estimation pipeline applied to DIPPR data for six cyclic ethers, ketones and a sulfide, and a comparison with the group-contribution SAFT- $\gamma$  Mie equation. Verification included exact reduction to SAFT-VR Mie at zero multipole moments ( $<10^{-13}$  relative), agreement with analytic Keesom mappings to  $(10^{-15})$ , automatic-differentiation checks to  $(10^{-11})$ , and the Clapeyron equation to  $(4 \times 10^{-11})$ . The fitted model reproduces vapour pressures, saturated liquid densities and vaporisation enthalpies of the weakly polar solvents within 0.1-1%, but its leading-order polar term fails for ketones unless an effective dipole moment is fitted, a diagnosis the LLM reached and explained. The LLM also located a sign inconsistency in a 2026 paper and a bug in Clapeyron.jl. We document the errors it made and how they were caught. We conclude that LLMs can now produce working, non-trivial thermodynamic models, while the researcher's effort shifts from production to verification.
\end{abstract}

\section{Introduction}

Large language models (LLMs) have become routine assistants for literature review, code completion and scientific writing. It is less clear whether they can contribute to the substance of science: deriving a new theory, implementing it correctly, checking it against exact limits, and drawing sound physical conclusions from data. Molecular thermodynamics offers an unusually demanding and unusually well-instrumented test of this question. Equations of state (EoS) based on the statistical associating fluid theory (SAFT) are derived from statistical mechanics, involve long perturbation expansions with empirical closures, require robust nonlinear solvers, and are validated against large experimental databases. A mistake anywhere in this chain
```

produces a model that still runs and still returns plausible numbers, which is precisely the failure mode most feared with LLMs.

In this paper we report a case study in which one researcher used one LLM, Claude Fable-5.1, to develop a new SAFT equation of state from a written brief. The target was a SAFT-VR variant in which segments interact through the isotropic multipolar potential (IMP) of Müller and Gelb²⁰⁰³, a spherically averaged representation of dipole-dipole, dipole-quadrupole, quadrupole-quadrupole and induction interactions. Because the IMP is a sum of inverse-power (Sutherland) terms, it can be treated with the machinery recently developed for SAFT-VR Mie²⁰¹³ and generalised to arbitrary Sutherland sums in SAFT-VR Sum²⁰²⁶. We call the resulting model SAFT-VR IMP.

The study is organised as a narrative in three parts. First, SAFT equations of state are useful and physically grounded, so extending them to polar molecules is worthwhile (Sections~\ref{sec:saft} and~\ref{sec:imp}). Second, developing a new SAFT equation is hard, which makes it a discriminating test of what an LLM can do (Section~\ref{sec:hard}). Third, the outcome of the case study, both its successes and its failures, is evidence about the current capability of LLMs in science (Sections~\ref{sec:methods}--\ref{sec:limits}).

The contributions of this paper are: (i) a new equation of state, SAFT-VR IMP, with its derivation and an implementation in Clapeyron.jl²⁰²² that reduces exactly to SAFT-VR Mie in the non-polar limit; (ii) a verification suite of 97 tests exercising exact limits, automatic differentiation and thermodynamic consistency; (iii) parameters for six polar solvents fitted to DIPPR-801 data, with a diagnosis of when the IMP approximation breaks down; (iv) a comparison with SAFT- γ Mie on identical data; and (v) a documented record of what the LLM derived, coded, got wrong, and corrected. We write for two audiences, thermodynamicists new to LLMs and AI-for-science researchers new to SAFT, and define all symbols and acronyms at first use.

\section{Background: SAFT equations of state}

\label{sec:saft}

\subsection{The SAFT framework}

SAFT²⁰¹³ models a fluid as chains of spherical segments that may carry short-ranged association sites. The residual Helmholtz free energy A^{res} of N molecules at volume V and temperature T is written as a sum of contributions,

\begin{equation}

$$\frac{A}{Nk_{\mathrm{B}}T} = a^{\mathrm{ideal}} + a^{\mathrm{mono}} + a^{\mathrm{chain}} + a^{\mathrm{assoc}},$$

\label{eq:saft}

\end{equation}

where k_{B} is Boltzmann's constant, a^{ideal} is the ideal-gas term, a^{mono} is the free energy of the fluid of unbonded segments, a^{chain} accounts for bonding the segments into chains of m segments per molecule, and a^{assoc} accounts for association. The chain and association terms derive from Wertheim's first-order thermodynamic perturbation theory (TPT1) of associating fluids^{1984,1986}, in which the free energy of chain formation is expressed through the contact value of the pair distribution function of the monomer fluid. The monomer term is where SAFT variants differ most: the original SAFT used a hard-sphere reference with a dispersion correlation, PC-SAFT²⁰⁰¹ uses a hard-chain reference, and the SAFT-VR family¹⁹⁹⁷ applies Barker-Henderson (BH) high-temperature perturbation theory^{1967,1976} to a variable-range attractive potential.

SAFT-VR Mie²⁰¹³ takes the Mie potential as the segment-segment interaction,

\begin{equation}

```

u^{\mathrm{Mie}}(r) = C\varepsilon\left[\left(\frac{\sigma}{r}\right)^{\lambda_r} - \left(\frac{\sigma}{r}\right)^{\lambda_a}\right],
\quad
C = \frac{\lambda_r}{\lambda_r - \lambda_a}\left(\frac{\lambda_r}{\lambda_a}\right)^{\lambda_a/(\lambda_r - \lambda_a)},
\label{eq:mie}
\end{equation}

```

with segment diameter σ , well depth ε , and repulsive and attractive exponents λ_r and λ_a . The monomer free energy is expanded to third order in $\beta = 1/(k_{\mathrm{B}}T)$, $a^{\mathrm{mono}} = a^{\mathrm{HS}} + \beta a_1 + \beta^2 a_2 + \beta^3 a_3$, around a hard-sphere reference of temperature-dependent diameter $d(T)$, and the chain term uses a second-order expansion of the radial distribution function at contact. Each term in the Mie potential is a Sutherland potential $\varepsilon(\sigma/r)^n$, and the first- and second-order perturbation integrals are evaluated term by term with a mean-value-theorem closure calibrated against simulation. SAFT- γ Mie~\cite{papaioannou2014} is the group-contribution counterpart: molecules are built from functional groups, each with its own Mie parameters and shape factor, and the group parameters are transferable across molecules.

\subsection{Why SAFT equations are useful}

SAFT-type equations are used throughout academic and industrial thermodynamics because they combine a molecular basis with practical accuracy for difficult systems. Their explicit association term makes them the standard choice for hydrogen-bonding fluids such as water, alcohols and amines and for their mixtures with hydrocarbons, where cubic equations require empirical mixing rules. Their chain term makes them applicable to polymers and polymer solutions, including cloud-point and liquid-liquid behaviour, and to asymmetric mixtures of light gases with heavy hydrocarbons. Their parameters are physically bounded, so that mixtures of many components can be described with few or no binary interaction parameters, as illustrated for natural-gas mixtures of up to twelve components in the Clapeyron.jl paper~\cite{walker2022}. In process engineering they underpin solvent screening and design, for example for amine-based and physical-solvent CO₂ capture~\cite{macdowell2010}\todo{VERIFY} and in computer-aided molecular design where the same parameters are used for the fluid and for the process model~\cite{CITATION NEEDED}.

\subsection{From SAFT parameters to coarse-grained force fields}

A distinctive advantage of SAFT-VR Mie is that its parameters $(m, \sigma, \varepsilon, \lambda_r, \lambda_a)$ are, literally, the parameters of a Mie force field for a chain of m tangent spheres. In the top-down ‘‘SAFT- γ coarse-grained force field’’ approach~\cite{avendano2011,muller2014}, parameters regressed to macroscopic data such as vapour pressures and liquid densities are used directly in molecular dynamics simulations of the corresponding Mie chain fluid, which then predict interfacial, transport and structural properties that the equation of state does not give. This mapping works because the theory is an accurate description of the fluid it claims to describe: the perturbation terms, the BH diameter, the effective packing fraction and the contact values were all calibrated against simulations of Mie fluids, so that the equation of state and the simulation agree with each other to within a few percent for the same potential. Parameters fitted to experiment therefore define a consistent molecular model rather than a set of correlation coefficients. The converse also holds: potentials fitted to \emph{ab initio} pair energies can be inserted into the theory and give predictions of macroscopic properties, an approach pursued for argon and neon with Mie potentials~\cite{walker2022jcp} and, with sums of Sutherland terms, in SAFT-VR Sum~\cite{jervell2026}. This two-way consistency between theory and molecular model is the property we want to preserve when polar interactions are added.

\section{Why developing a new SAFT equation of state is difficult}

\label{sec:hard}

A new SAFT variant is not a single formula but a chain of coupled pieces, each of which can fail independently. We list the difficulties in the order they arise, and note in passing where they arose in the present study.

`\paragraph{Deriving new perturbation contributions.}`

Adding a new interaction to the segment potential changes a_1 , a_2 and the contact value of the radial distribution function. The derivation must be carried out consistently in the same reference system and with the same closures as the parent theory, otherwise the model does not reduce to its parent in the appropriate limit. In this study, for example, the first-order contact value g_1 required matching the Clausius virial route to the thermodynamic route order by order in β , and the sign of one term was found to be printed inconsistently in the SAFT-VR Sum paper~\cite{jervell2026} (Section~\ref{sec:imp}).

`\paragraph{Obtaining the required integrals and distribution functions.}`

Perturbation terms are integrals of the pair potential over the hard-sphere pair distribution function $g^{\mathrm{HS}}(r)$. SAFT-VR Mie replaces them by closed forms through a mean-value theorem with an effective packing fraction $\zeta_{\mathrm{eff}}(\zeta_x, \lambda)$ correlated against simulation, plus short-range corrections that use the contact value and slope of g^{HS} . For a new potential one must decide whether these correlations, fitted for Mie exponents between about 5 and 100, remain valid. When the potential is temperature dependent, the BH diameter itself is an integral whose integrand and upper limit move with T .

`\paragraph{Thermodynamic consistency and derivative properties.}`

All properties follow from derivatives of $A(V, T, \mathbf{z})$: pressure and chemical potentials from first derivatives, heat capacities and speed of sound from second derivatives, critical points from third derivatives with respect to volume and composition. An equation of state with a temperature-dependent potential is a free energy, not an energy, so every temperature derivative must include $\partial u / \partial T$; omitting it silently violates the Clapeyron equation. Modern packages obtain the derivatives by automatic differentiation (AD)~\cite{revels2016}, which requires that every intermediate quantity, including the outputs of iterative solvers, propagate derivative information correctly.

`\paragraph{Robust phase-equilibrium solvers.}`

Saturation pressures, critical points and bubble/dew points are roots of nonlinear systems that need good initial guesses. Solvers tuned for one family of models can fail for another, and during global parameter optimisation they are called with thousands of physically unreasonable trial parameters. In this study a latent bug in the saturation-pressure retry logic of Clapeyron.jl was exposed only under these conditions (Section~\ref{sec:fitting}).

`\paragraph{Parameter estimation.}`

Pure-component parameters are typically regressed to vapour pressures and liquid densities. The objective is multimodal, several parameters are strongly correlated (ϵ with σ^3 , λ_r with ϵ , and any polar parameter with the dispersion energy), and the available data rarely cover the near-critical region. Global search followed by local refinement is required, and bounds must be chosen so that the physical meaning of the parameters is retained.

`\paragraph{Validation across compounds.}`

A model that fits one compound proves little. It must be tested on chemically distinct molecules, compared with established models on identical data, and its failures must be diagnosed in terms of the underlying physics rather than absorbed into parameters.

Each of these steps takes an experienced researcher days to weeks, and each produces artefacts (long derivations, hundreds of lines of code, thousands of data points) that are expensive to audit. This is what makes the task a stringent test of an LLM: the cost of producing plausible output is low, while the cost of producing correct output is high and the difference is not visible without deliberate verification.

\section{Modelling polar and dipolar molecules with the IMP}

\label{sec:imp}

\subsection{Limitations of implicit treatments}

Most SAFT parameterisations of polar fluids treat polarity implicitly. Dipolar attraction is absorbed into the dispersion energy ϵ and, for strongly polar or hydrogen-bonding molecules, into association parameters attached to fictitious sites. This works for pure-component vapour pressures and densities, but it distorts the meaning of the parameters: the fitted ϵ no longer corresponds to the dispersion energy of a coarse-grained force field, the temperature dependence of polar attraction (which scales as $1/T$ for dipoles) is wrong, and mixtures of polar with non-polar components typically need large binary interaction parameters to compensate. Explicit polar terms exist in several SAFT variants: the Jog--Chapman--\cite{jog1999} and Gross--Vrabec--\cite{gross2006} dipolar terms are third-order perturbation expansions of the multipolar Helmholtz energy with segment-based reduced moments $\mu^{*2} = \mu^2/(m\epsilon\sigma^3)$, and analogous terms have been attached to SAFT-VR Mie--\cite{CITATION NEEDED}. These terms add a separate free-energy contribution on top of the monomer term.

\subsection{The isotropic multipolar potential}

The IMP of Muller and Gelb--\cite{muller2003} takes a different route: it folds the polar interactions into the pair potential itself. For two axially symmetric multipolar molecules at separation r and relative orientation ω , the orientationally averaged Boltzmann factor defines an isotropic potential of mean force $\Gamma(r;T)$ through $e^{-\beta\Gamma} = \langle e^{-\beta u(r,\omega)} \rangle_\omega$. A cumulant expansion gives $\beta\Gamma = \beta \langle u \rangle_\omega - \frac{1}{2}\beta^2 \langle u^2 \rangle_\omega + O(\beta^3)$, and because the unweighted angular average of permanent multipole--multipole interactions vanishes,

\begin{equation}

$$\Gamma^\mu = -\frac{\beta \mu_k^2 \mu_l^2}{3r^6}, \quad$$

$$\Gamma^Q = -\frac{\beta (\mu_k^2 Q_l^2 + Q_k^2 \mu_l^2)}{2r^8}, \quad$$

$$\Gamma^{QQ} = -\frac{7\beta Q_k^2 Q_l^2}{5r^{10}},$$

\label{eq:imp}

\end{equation}

for dipole moments μ and quadrupole moments Q (Gaussian units; in SI $\mu^2 \rightarrow \mu^2/4\pi\epsilon_0$), together with the orientationally averaged dipole--induced-dipole (Debye) term $\Gamma^\alpha = -(\mu_k^2 \alpha_l + \alpha_k \mu_l^2)/r^6$ for polarisability α , which carries no factor of β . The first term is the classical Keesom potential--\cite{CITATION NEEDED}. Two properties of these expressions matter for what follows. First, they are inverse-power terms, so the IMP is a Sutherland sum and the SAFT-VR machinery applies without new integrals. Second, they are the leading term of an expansion in β : the exact orientational free energy of a dipole pair is $-k_B T \ln(1+x)$ with $x = \mu_k^2 \mu_l^2 / (3k_B T^2 r^6)$ (eq.~10 of Muller and Gelb), which is always smaller in magnitude than its linearisation $-k_B T x$. The IMP therefore overstates polar attraction when x at contact approaches unity, that is, for strong dipoles at low temperature. We use $x(\sigma)$ evaluated at the normal boiling point as a validity indicator below.

Muller and Gelb combined the IMP with a Lennard-Jones core and used it in large temperature-quench molecular dynamics simulations, reproducing the coexistence curves of benzene and naphthalene, the high-pressure PVT behaviour of CO₂, the azeotropic VLE of 1,2-dichloroethane + cyclohexane and the liquid--liquid equilibrium of sulfolane + n -octane + benzene with only two adjustable parameters per compound. Their work showed that the average energetics captured by the IMP dictate the gross features of fluid-phase behaviour. Turning the IMP into an equation of state lets the same potential be used analytically.

\subsection{SAFT-VR IMP}

The segment--segment potential of SAFT-VR IMP, as formulated in this study, adds a Mie core to the IMP and writes the result as a five-term Sutherland sum,

```
\begin{equation}
u_{kl}(r;T) = \sum_{n=1}^5 c_{kl,n}(T) \left( \frac{\sigma_{kl}}{r} \right)^{\lambda_n},
\label{eq:sum}
\end{equation}
with exponents and coefficients (energies in kelvin, i.e.  $u/k_{\mathrm{B}}$ )
\begin{center}
\begin{tabular}{c}
\toprule
 $n$  &  $\lambda_n$  &  $c_{kl,n}(T)$  \\
\midrule
1 &  $\lambda_{r,kl}$  &  $C_{kl} \epsilon_{kl}$  \\
2 &  $\lambda_{a,kl}$  &  $-C_{kl} \epsilon_{kl}$  \\
3 & 6 &  $-\left[ \hat{\mu}_k^2 \hat{\alpha}_l + \hat{\alpha}_k \hat{\mu}_l^2 + \right. \\
& & \left. \hat{\mu}_k^2 \hat{\mu}_l^2 / (3T) \right] / \sigma_{kl}^6$  \\
4 & 8 &  $-(\hat{\mu}_k^2 \hat{Q}_l^2 + \hat{Q}_k^2 \hat{\mu}_l^2) / (2T \sigma_{kl}^8)$  \\
5 & 10 &  $-7 \hat{Q}_k^2 \hat{Q}_l^2 / (5T \sigma_{kl}^{10})$  \\
\bottomrule
\end{tabular}
\end{center}
```

The molecular moments are distributed uniformly over the m_i segments of a chain as $\hat{\mu}_i^2 = \mu_i^2 / (4\pi \epsilon_0 m_i)$, $\hat{Q}_i^2 = Q_i^2 / (4\pi \epsilon_0 m_i)$ and $\hat{\alpha}_i = \alpha_i / m_i$, so that the far field of the m_{kl} segment pairs reproduces the molecular interaction; this is the same scaling as the reduced moments of the Gross--Vrabec terms, and recovers the Muller--Gelb potential exactly for $m=1$. Numerically, $(1/\epsilon_{\mathrm{D}})^2 / (4\pi \epsilon_0 k_{\mathrm{B}}) = 7242.97 \text{ K-Å}^3$. The Mie part uses the SAFT-VR Mie combining rules (arithmetic σ_{kl} , Hudson--McCoubrey ϵ_{kl} with an optional k_{kl} , and $\lambda_{kl}-3 = \sqrt{(\lambda_k-3)(\lambda_l-3)}$), while the multipolar coefficients need no combining rule because they are products of pure-component moments. This is why the model stores μ , Q and α rather than a list of Sutherland energies: the dipole--quadrupole cross coefficient of a mixture can be non-zero when both like-pair coefficients at $\lambda=8$ vanish, and no combining rule on the energies could generate it.

Following SAFT-VRQ Mie--\cite{aasen2019} and SAFT-VR Sum--\cite{jervell2026}, the theory is built on effective quantities of the full potential: σ_{eff} with $u(\sigma_{\mathrm{eff}}) = 0$, the well depth $\epsilon_{\mathrm{eff}} = -\min_r u$, and the dimensionless van der Waals energy

```
\begin{equation}
\alpha = -\frac{1}{\epsilon_{\mathrm{eff}} \sigma_{\mathrm{eff}}^3} \int_{\sigma_{\mathrm{eff}}}^{\infty} u(r) r^2 dr
= -\frac{1}{\epsilon_{\mathrm{eff}}} \sum_n \frac{c_n}{\lambda_n-3} \left( \frac{\sigma_{\mathrm{eff}}}{\sigma_{\mathrm{eff}}} \right)^{\lambda_n}.
\end{equation}
```

With $x = r/\sigma$ and $s = \ln x$, both the zero and the minimum of the potential are roots of a function of the form $F(s) = \ln w_1 - \lambda_1 s - \ln \sum_{n \geq 2} w_n e^{-\lambda_n s}$ with positive weights, which is strictly decreasing and concave, so that Newton's method converges globally from the Mie solution. The BH diameter is $d_k(T) = \int_0^\infty \sigma_{\mathrm{eff},kk} [1 - e^{-\beta u_{kk}}] ds$, evaluated by the same Gauss--Laguerre/Gauss--Legendre strategy as SAFT-VR Mie. The monomer free energy per segment is

```
\begin{equation}
a^{\mathrm{mono}} = \bar{m} \left[ a^{\mathrm{HS}} + \beta a_1 + \beta^2 a_2 + (\beta^3 a_3) \right],
\end{equation}
```

where $\bar{m} = \sum_i x_{im_i}$, a^{HS} is the Boublík--Mansoori--Carnahan--Starling--Leland term, and, defining the SAFT-VR Mie kernels $a_1^{\mathrm{S}}(\lambda)$ (mean-value-theorem integral from d to ∞) and

$B(\lambda, x_{\mathrm{eff}})$ (short-range correction from d to σ_{eff}) with $\Phi(\lambda) = a_1 S(\lambda) + B(\lambda, x_{\mathrm{eff}})$, $x_0 = \sigma/d$, $x_{\mathrm{eff}} = \sigma_{\mathrm{eff}}/d$,

$$\begin{aligned} a_{1,kl} &= -2\pi\rho_s d_{kl}^3 \sum_n c_{kl,n} \backslash, x_0^{\lambda_n} \backslash, \Phi(\lambda_n), \backslash \\ a_{2,kl} &= \pi\rho_s K^{\mathrm{HS}} \\ (1+\chi_{kl}) \backslash, d_{kl}^3 \sum_{n,m} c_{kl,n} c_{kl,m} \backslash, x_0^{\lambda_n+\lambda_m} \backslash, \Phi(\lambda_n+\lambda_m), \end{aligned}$$

with ρ_s the segment density, K^{HS} the hard-sphere isothermal compressibility and $\chi(\alpha, \bar{z}_x)$ the Lafitte correction. The second-order term runs over the 15 unordered pairs of the five Sutherland terms. The optional third-order term is Lafitte's correlation evaluated with $\varepsilon_{\mathrm{eff}}$ and α . The chain term is TPT1 with the contact taken at σ_{eff} , where $u = 0$ and the cavity function equals the pair distribution function:

$$\begin{aligned} a^{\mathrm{chain}} &= -\sum_i x_i(m_i-1) \ln y_{ii}(\sigma_{\mathrm{eff}}, ii), \quad \\ \ln y &= \ln g^{\mathrm{HS}}_d(x_{\mathrm{eff}}) + \frac{\tau g_1 + \tau^2 g_2}{g^{\mathrm{HS}}_d(x_{\mathrm{eff}})}, \quad \tau = \beta \varepsilon_{\mathrm{eff}}, \end{aligned}$$

with Boublík's expression for the hard-sphere distribution function away from contact,

$$\begin{aligned} g_1 &= \frac{1}{\varepsilon_{\mathrm{eff}}} \Big[\sum_n \lambda_n c_{nx} x_0^{\lambda_n} \Phi(\lambda_n) - \\ &3 \frac{\partial(\rho_s S_1)}{\partial \rho_s} \Big], \quad S_1 = \sum_n c_{nx} x_0^{\lambda_n} \Phi(\lambda_n), \end{aligned}$$

and $g_2 = (1+\gamma_c) g_2^{\mathrm{MCA}}$ from the macroscopic compressibility approximation. The full derivation is given in Appendix~A.

Three choices distinguish SAFT-VR IMP from the original brief and deserve comment. The brief specified a hard core; the delivered model uses a soft Mie core because a hard core gives $d = \sigma$ with no temperature dependence, because Müller and Gelb themselves used an r^{-12} core, and because the hard-core chain term requires a non-perturbative Boltzmann factor at contact. The brief specified a second-order expansion, which is the default (`order = 2`); a third-order option (`order = 3`) adds a_3 and makes the model identical to SAFT-VR Mie when all moments vanish, which provides an exact test. Finally, the tail term in g_1 enters with a minus sign, as in eq.~(29) of SAFT-VR Sum~\cite{jervell2026} and in Lafitte's g_1 ; the plus sign printed in eq.~(32) of that paper is inconsistent with its eq.~(29), and only the minus sign reproduces SAFT-VR Mie. When the potential is written as $\varepsilon \sum_k c_k (\sigma/r)^{\lambda_k}$ with g_n scaled by $\varepsilon_{\mathrm{eff}}^n$, the tail term also carries a factor $\varepsilon/\varepsilon_{\mathrm{eff}}$.

How does SAFT-VR IMP describe polar molecules better than an implicit treatment? The polar attraction is separated from dispersion, carries its own $1/T$ dependence, and enters the pair potential with the experimental dipole and quadrupole moments, so that ε , σ and λ_r retain their meaning as Mie force-field parameters and the resulting potential can be used directly in simulation. Cross interactions in mixtures are predicted from pure-component moments with no additional parameters. The price is the leading-order truncation discussed above; its consequences are the central empirical finding of this study.

`\section{Methods: LLM-assisted development}`

`\label{sec:methods}`

`\subsection{The LLM and the workflow}`

All LLM work was performed with Claude Fable-5.1 (Anthropic) through the claude.ai web interface, which provided the model with a Linux sandbox (one CPU) in which it could run shell commands, create and edit

files, and execute code. The researcher supplied three PDF papers~\cite{muller2003,jervell2026,walker2022} and a zip archive of the Clapeyron.jl source (master branch, v0.6.28), and issued three prompts over three sessions:

```
\begin{enumerate}
\item A structured brief asking for the formulation and implementation of a SAFT-VR variant with the IMP as a Sutherland sum, with the mathematical constraints, data structures and validation script listed in Section~\ref{sec:imp}.
\item Six DIPPR-801 spreadsheet exports and the instruction ``Write a Clapeyron code to fit the equation of state of SAFT-VR IMP to the following experimental data.''
\item The question ``How do the results of SAFT-VR IMP compare with SAFT-gamma Mie?''
\end{enumerate}
```

In addition, the researcher typed ``Continue'' four times when the LLM's turn was interrupted by tool or output limits. No other guidance, correction or code was supplied by the researcher during the sessions; all corrections described below were made by the LLM in response to its own tests. The researcher's role was that of a principal investigator reviewing reports.

In the first session the LLM installed Julia 1.11.7 in the sandbox, read the SAFT-VR Mie, SAFT-VRQ Mie and shared SAFT source files of Clapeyron.jl, studied the macro system used to define models, wrote the derivation, wrote the model file, wrote and ran a test script, wrote a Markdown derivation document, generated an in-tree patch and verified that it applied and precompiled. In the second session it parsed the DIPPR files with Python, studied Clapeyron's \texttt{Estimation} module, installed Metaheuristics.jl~\cite{mejia2022}\todo{VERIFY}, wrote and ran the fitting script, and analysed the results. In the third it identified the available SAFT- γ Mie groups, wrote and ran a comparison script, and checked the group assignments against cyclohexane and cyclopentane.

\subsection{Implementation}

The implementation is a single Julia file of about 600 lines following Clapeyron.jl conventions: a parameter struct \texttt{SAFTVRIMPParam}\{T\} <: ParametricEoSParam\{T\} holding M_w , m , σ , λ_a , λ_r , ε , μ , Q and α ; a model struct \texttt{SAFTVRIMP} <: \texttt{SAFTVRIMPModel} <: \texttt{SAFTModel} with an ideal model and a non-splittable options struct carrying the perturbation order; a constructor reading CSV databases or user-supplied named tuples; and the functions \texttt{a_hs}, \texttt{a_disp}, \texttt{a_mono}, \texttt{a_chain} and \texttt{a_res}. The dispersion and chain terms are evaluated in one fused pass (\texttt{imp_dispchain}) as in Clapeyron's SAFT-VR Mie, because they share the same kernels. Temperature-dependent pair data (λ_n , c_n , an activity mask, σ_{eff} , $\varepsilon_{\mathrm{eff}}$, α , d) are computed once per state in \texttt{imp_pairdata}.

Four design decisions keep the model compatible with forward-mode AD via ForwardDiff.jl~\cite{revels2016}: arrays are typed with the promoted element type of the model and $\mathbb{T}$ so that dual numbers are never truncated; the mask of active Sutherland terms is decided from the Float64 parameters, never by comparing dual values; the implicit solves for σ_{eff} and $r_{\min}$ iterate Newton on the primal value and then take three additional Newton steps at the converged root, each propagating one further order of dual parts, so that nested derivatives (heat capacities, speed of sound) are exact; and the quadrature for $d(T)$ uses fixed rules whose nodes depend smoothly on $\mathbb{T}$. The model can be loaded without modifying Clapeyron (\texttt{Base.include(Clapeyron, "SAFTVRIMP.jl")}) or added in-tree with a patch. Evaluation cost is close to SAFT-VR Mie: 1.7~ μ s versus 1.3~ μ s per pressure call for n -hexane, 3.1~ μ s for quadrupolar benzene, and 14.2~ μ s for a dipolar binary mixture.

\subsection{Verification: what ``working'' means}

\label{sec:verification}

We define a working equation of state as one that (i) runs end to end in the host package, including

construction from a database, bulk properties, saturation pressure, critical point and mixture bubble points; (ii) reduces exactly to its parent model in the appropriate limit; (iii) yields derivatives that agree with finite differences and satisfy thermodynamic identities; and (iv) reproduces its reported results when rerun. The LLM wrote a test script of 97 assertions covering these points, which we reran on the final code with identical numbers. Table~\ref{tab:verif} summarises the checks. Two are worth emphasising for readers new to SAFT. The reduction test compares SAFT-VR IMP with all moments set to zero and \texttt{order = 3} against Clapeyron's independently written \texttt{SAFTVRMie} at liquid, vapour and supercritical states, including a state at 1200-K that exercises the Gauss--Legendre branch of $d(T)$; agreement to better than 10^{-13} in a^{res} , p and μ^{res} for n -hexane and n -hexane + n -decane, and to all printed digits in the normal boiling point (341.86441622-K), verifies the reference term, both perturbation terms, the chain term (including the contested sign of g_1) and the mixing rules at once. The Clapeyron-equation test, $\frac{dp_{\mathrm{sat}}}{dT} = \frac{\Delta h_{\mathrm{vap}}}{T\Delta v}$ for quadrupolar benzene at 450-K, satisfied to 4×10^{-11} , is a test of the physics as much as of the code: it holds only if the enthalpy contains the $\partial u/\partial T$ contributions of the temperature-dependent multipolar terms.

Verification tests written and run by the LLM (97 assertions, all passing). Tolerances are relative.	
Test & Result	
Zero moments, order 3, vs Clapeyron \texttt{SAFTVRMie}: a^{res} , p , μ^{res} (n -hexane; n -hexane + n -decane, four states incl. 1200-K); T_b ; $a^{\mathrm{mono}}+a^{\mathrm{chain}}=a^{\mathrm{res}}$ & $<10^{-13}$; T_b identical to 10^{-9} \\ Dipolar LJ fluid ($\mu^2=2$) vs analytic Keesom mapping: σ_{eff} , $\varepsilon_{\mathrm{eff}}$, d , a^{res} , p at 120--3000-K; $d\varepsilon_{\mathrm{eff}}/dT$, $d^2\varepsilon_{\mathrm{eff}}/dT^2$ & $\leq 5\times 10^{-15}$; derivatives to 13 digits \\ $\lambda=6,8,10$ slots, chains ($m=1.5,2$), explicit unlike parameters vs analytically mapped \texttt{SAFTVRMie} & $\leq 7\times 10^{-14}$ \\ AD vs Richardson finite differences, polar chain + decane, all five terms active, order 2: $\partial p/\partial T$, $\partial p/\partial V$, $\partial p/\partial z_1$, $\partial^2 a^{\mathrm{res}}/\partial T^2$; C_v vs $\partial U/\partial T$; Euler relation; dx_{eff}/dT , $d\varepsilon_{\mathrm{eff}}/dT$ vs implicit-function and envelope theorems & $\leq 4\times 10^{-11}$; 6×10^{-12} ; 13 digits \\ Clapeyron equation, benzene (Muller--Gelb parameters), 450-K & 4×10^{-11} \\ n -hexane order 2: $p(V,T)$, gradient, saturation curve 300--500-K, T_b , T_c & runs; $T_c^{(2)}>T_c^{(3)}$ \\ In-tree patch applies to pristine tree and precompiles & yes \\	

Beyond exact limits, the LLM ran the model with the Muller--Gelb simulation parameters (Table~1 of ref.~\cite{muller2003}, moments converted to D and $D\backslash AA$) as a plausibility check; these numbers are reported in Section~\ref{sec:results}.

Parameter estimation

fitting

The experimental data were six DIPPR-801 exports for tetrahydrofuran (THF), tetrahydropyran (THP), 2-

butanone, cyclohexanone, cyclopentanone and tetrahydrothiophene (THT), all polar and non-associating. Because Julia has no reader for the legacy `\texttt{xls}` format, a Python script converts each workbook to CSV, preserving DIPPR's data type, acceptance code, uncertainty class and dataset identifier for every raw point, together with the compound constants and the DIPPR correlations (equations 101, 105 and 106). The DIPPR gas-phase dipole moments are 1.631, 1.550, 2.779, 3.088, 3.238 and 1.901-D respectively.

The fitted properties were saturated vapour pressure, saturated liquid density and enthalpy of vaporisation, using points flagged Experimental and Accepted with $T \leq 0.9T_c$ (SAFT has classical critical behaviour, so near-critical data are excluded by convention) and DIPPR uncertainty classes up to 5, 3 and 5\,\%. Repeated measurements at the same temperature (± 0.01 K) were averaged (geometric mean for pressures). The data are unevenly distributed: accepted liquid densities reach only about $0.6T_c$ for every solvent, the accepted THP vapour pressures span only $0.52T_c$ – $0.63T_c$, and cyclopentanone has only two accepted vaporisation-enthalpy points. Four parameters were adjusted, $\ln[1,4]$, $\sigma_{[3,5]}$, $\epsilon/k_B \ln[150,700]$ K and $\lambda_r \ln[8,30]$, with $\lambda_a=6$ fixed, $Q=0$, and $\alpha=0$ (for a pure fluid an induction term is an r^{-6} contribution indistinguishable from dispersion). The objective is Clapeyron's relative mean squared error per property, $\sum_{pw,p} N_p^{-1} \sum_i [(y_i^{\text{calc}} - y_i^{\text{exp}})/y_i^{\text{exp}}]^2$ with weights 1 : 1 : 0.5, minimised by the evolutionary centres algorithm (ECA, population 40, 150 iterations, fixed seed) of `Metaheuristics.jl` and polished by Nelder–Mead–[cite{nelder1965}](#) in logistically bound-transformed variables. A failed saturation calculation for a trial parameter set is counted as a 100\,\% deviation rather than aborting the trial. Each compound took 31–166 s per variant on one CPU.

Global optimisation exposed a bug in the `Clapeyron.jl` master used: in `\texttt{src/methods/initial\_guess.jl}`, the critical-point retry of `\texttt{x0\_sat\_pure(model,T,crit)}` calls `\texttt{x0\_sat\_pure\_crit(satmodel)}` without its `T` and `crit` arguments and throws a `MethodError` whenever a first saturation attempt fails above $0.6T_c$, which random trial parameters trigger constantly. The fitting script overrides the method for SAFT-VR IMP models; the upstream fix is one line.

\subsection{The effective-dipole suggestion}

With the experimental dipole moments fixed, the fits for the three ketones ended on the parameter bounds ($\sigma=5$ – ∞ for all three, $\lambda_r=8$ for the cyclic ketones) and their deviations followed systematic temperature trends rather than scatter: for 2-butanone the vapour pressure was 6.7\,\% low at 273-K, about 1\,\% high at intermediate temperatures and 8.9\,\% low at 483-K, the liquid expanded too little with temperature, and Δh_{vap} was 2–3.5\,\% high. The LLM attributed this to the leading-order truncation of the IMP: with realistic geometry, the contact value of $x = \hat{\mu}^4/(3T^2\sigma^6)$ is 0.64 for cyclopentanone at its boiling point and 1.4 at 273-K, so the linearised polar term overstates the attraction by 30–60\,\%, and the optimiser compensates by inflating σ . It proposed, citing Muller and Gelb's recommendation for strongly polar species, to fit an effective dipole moment $\mu_{\text{eff}} \in [0, \mu_{\text{exp}}]$ as a fifth parameter.

The physical rationale for an effective dipole is broader than the truncation argument used in the sessions, and we add it here. A real molecule's charge distribution is not a point dipole at the segment centre: higher multipoles, the finite extent of the charges relative to σ , and the distribution of the moment over m segments all modify the effective interaction at contact. In the condensed phase, mutual polarisation enhances the average dipole (liquid-phase dipole moments of polar molecules are typically larger than gas-phase values), whereas the IMP truncation and orientational correlations beyond the pair level reduce the effective interaction. The net effect is not known a priori, and an adjustable μ_{eff} absorbs it. The trade-offs are also clear. An extra parameter adds flexibility that is partly degenerate with ϵ and σ : the LLM observed that for the ethers and the sulfide, where the model is valid, freeing the dipole improved the objective by at most 13\,\% while μ_{eff} scattered between 0 and 1.55-D depending on compound and perturbation order (1.55-D for THF at order 3 but 0.85-D at order 2), with ϵ and σ moving along a degenerate valley. And an effective dipole far from the

experimental value destroys the transferability that motivated the model: the cross interactions in mixtures are then computed from a moment with no physical meaning. Both fixed-dipole and effective-dipole fits were carried out for all six compounds at both perturbation orders, and the results are compared in Section~\ref{sec:results}.

\subsection{Error record}
\label{sec:errors}

We list the errors made by the LLM during the three sessions, how they were detected, and by whom. None was detected by the researcher.

\begin{itemize}
\item \emph{Numerical.} The convergence tolerance of the Newton solve for σ_{eff} was initially $4\epsilon_{\mathrm{machine}}$, which the iteration cannot always reach; the LLM relaxed it to 10^{-12} after the test run, noting that the three polishing steps restore machine precision. Detected by the LLM's own test run.
\item \emph{Data parsing.} The first parser of the DIPPR raw-data sheet mistook ``Notes:'' rows for property headers, and the second mis-assigned acceptance codes because the metadata rows share rows with the first data points of each block. Detected by the LLM comparing per-property counts with a block-level survey.
\item \emph{Code listings.} The Julia listing pasted into the chat after the second session (when the deliverables had not yet been copied out of the sandbox) contained a parse error on its last line (`@__FILE__ \&\& main()`), where the macro swallows the conjunction), and the Python listing used a backslash inside an f-string, which only Python~3.12 accepts. Both listings had been restructured from the tested versions, and the LLM flagged several parts as untested at the time. In the third session it saved the pasted listings verbatim, ran them, found both errors, and fixed them. A latent bug in an untested weighting branch (`minimum(..., init=NaN)` returns NaN for every input) was found in the same way.
\item \emph{Incomplete suggestion.} The first version of the effective-dipole modification was a code snippet that, if applied, would have shifted the columns of the summary file and written the experimental dipole into the parameter file. Replaced by a complete, tested implementation in the next session.
\item \emph{Environment.} Background jobs did not survive between turns, a file-creation call was aborted by the tool layer, and tool and output limits interrupted delivery twice, once leaving the deliverables in the sandbox rather than in the output folder. These are infrastructure failures rather than reasoning errors, but they cost turns and required the LLM to re-verify state at the start of each session.
\end{itemize}

We also record what the LLM did \emph{not} do. It did not derive the ζ_{eff} , χ , a_3 or γ_c correlations, which are taken from Lafitte et al.\ through the α -hypothesis, nor did it run molecular simulations of the IMP fluid to test the perturbation theory itself. It did not fit SAFT-VR Mie without a polar term to the six solvents as an explicit baseline, and it did not test the fitted parameters in mixtures. No association term was implemented. These are noted as future work in Section~\ref{sec:conclusion}.

[Stopping at the end of Section 6. Reply "continue" for Sections 7-10 and the appendices.]

9/17/2026, 9:24:41 AM



continue

9/17/2026, 9:24:52 AM



\section{Results}
\label{sec:results}

\subsection{Behaviour with the M\uller--Gelb parameters}

Before fitting, the LLM evaluated SAFT-VR IMP with the parameters that M\uller and Gelb regressed for their simulations (Lennard-Jones core, moments at experimental values, ϵ and σ fitted to liquid densities or to the LJ corresponding-states critical point of a potential cut and shifted at 5σ). These parameters were not fitted to the equation of state, so the results test plausibility rather than accuracy. Table~\ref{tab:mg} lists the predicted critical temperatures. The third-order model reproduces the LJ critical temperature $T_c^* = 1.3122$ (the non-polar fits used $T_c^* = 1.31$), and the second-order truncation raises it by about 3.7\%; for the dipolar LJ fluid of M\uller and Gelb's Fig.~3 ($\mu^2=2$) the model gives $T_c^* = 1.833$ (order 3) and 1.885 (order 2), above the highest two-phase simulation state they report ($T^* = 1.7$). Switching the moments off lowers T_c of benzene by 83~K and of CO₂ by 49~K at order 2, which shows that the quadrupolar terms carry a large share of the cohesive energy in this representation. With pure-component parameters only, the model predicts an azeotrope for 1,2-dichloroethane + cyclohexane at 1~atm, as observed, but the pure boiling points are 25--45~K too low (329~K and 310~K), the same vapour-pressure failure that M\uller and Gelb reported for these liquid-density-fitted parameters.

```
\begin{table}[t]
\centering
\caption{Critical temperatures (K) predicted by SAFT-VR IMP with the M\uller--Gelb simulation parameters.
``moments off'': dipole and quadrupole set to zero.}
\label{tab:mg}
\begin{tabular}{lcccc}
\toprule
Fluid & order 2 & order 3 & order 2, moments off & experiment \\
\midrule
benzene ($Q$) & 571.5 & 553.5 & 488.2 & 562.0 \\
CO2 ($Q$) & 341.7 & 331.0 & 292.6 & 304.1 \\
1,2-dichloroethane ($\mu$) & 599.2 & 578.9 & 568.6 & 561.6 \\
cyclohexane & 560.6 & 540.6 & 560.6 & 553.6 \\
n$-octane & 588.2 & 567.2 & 588.2 & 568.7 \\
\bottomrule
\end{tabular}
\end{table}
```

\subsection{Fitted parameters for six polar solvents}

Table~\ref{tab:params} gives the order-3 parameters with the experimental dipole (the recommended sets for the ethers and the sulfide) and with the fitted effective dipole (the only usable sets for the ketones), and Table~\ref{tab:aad} the corresponding average absolute deviations (AAD\%) over the fitted points, the deviations from the DIPPR correlations between $\max(T_{\text{tp}}), 0.45T_c$ and $0.9T_c$ as an extrapolation check, and the critical-point errors. Order-2 results are given in the supplementary data files; for the fixed-dipole fits they differ from order 3 mainly in the critical temperature, which the second-order truncation overshoots by roughly twice as much (e.g. THF \$+6.2\$ versus \$+3.1\%\$, THP \$+3.8\$ versus \$+1.6\%\$) at essentially the same AADs.

```

\begin{table}[t]
\centering
\caption{SAFT-VR IMP parameters (order 3,  $\lambda_a=6$ ,  $Q=\alpha=0$ ) fitted to DIPPR data. Asterisks mark parameters that ended on a search bound.  $\mu_{\text{exp}}$  is the DIPPR dipole moment.}
\label{tab:params}
\small
\begin{tabular}{llccccc}
\toprule
Solvent & dipole &  $m$  &  $\sigma/\text{\AA}$  &  $\epsilon/k_{\text{B}}$  &  $\lambda_r$  &  $\mu/D$  \\
( $\mu_{\text{exp}}$ ) & & & & & & \\
\midrule
tetrahydrofuran & experimental & 1.5708 & 4.2123 & 385.03 & 14.715 & 1.631 \\
& effective & 1.6048 & 4.1793 & 385.56 & 14.915 & 1.548 (1.631) \\
tetrahydropyran & experimental & 2.0005 & 4.1044 & 328.12 & 12.320 & 1.550 \\
& effective & 2.2219 & 3.9434 & 321.28 & 12.594 & 0.000* (1.550) \\
tetrahydrothiophene & experimental & 1.7570 & 4.2071 & 370.68 & 11.656 & 1.901 \\
& effective & 1.4255 & 4.5895 & 552.51 & 19.814 & 0.972 (1.901) \\
2-butanone & experimental & 1.1035 & 5.000* & 372.13 & 11.844 & 2.779 \\
& effective & 1.9065 & 4.0864 & 409.99 & 18.260 & 0.002* (2.779) \\
cyclohexanone & experimental & \todo{MISSING: order-3 fixed-dipole parameters not printed} & & & & 3.088 \\
& effective & 1.6553 & 4.6218 & 552.47 & 19.929 & 0.319 (3.088) \\
cyclopentanone & experimental & 1.2907 & 5.000* & 272.43 & 8.000* & 3.238 \\
& effective & 1.7935 & 4.2290 & 488.16 & 18.213 & 0.333 (3.238) \\
\bottomrule
\end{tabular}
\end{table}

```

```

\begin{table}[t]
\centering
\caption{Average absolute deviations (%) of SAFT-VR IMP (order 3) from the fitted DIPPR points ( $\rho_{\text{sat}}$  /  $\rho_L$  /  $\Delta h_{\text{vap}}$ ), from the DIPPR correlations over 0.45-0.9,  $T_c$  (extrapolation check), and critical-point errors.  $x(T_b)$  is the IMP validity indicator with the dipole used in the fit.}
\label{tab:aad}
\small
\begin{tabular}{llcccccc}
\toprule
Solvent & dipole & AAD fitted points & AAD vs DIPPR corr. &  $T_c$  dev. &  $P_c$  dev. &  $x(T_b)$  \\
\midrule
THF & exp. & 0.46 / 0.22 / 0.89 & 0.50 / 1.04 / 2.09 &  $\pm 3.1$  &  $\pm 19.6$  & 0.079 \\
& eff. & 0.44 / 0.22 / 0.87 & 0.42 / 0.98 / 2.07 &  $\pm 2.9$  &  $\pm 19.1$  & \todo{MISSING} \\
THP & exp. & 0.51 / 0.08 / 0.20 & 0.99 / 0.79 / 0.87 &  $\pm 1.6$  &  $\pm 7.4$  & 0.086 \\
& eff. & 0.49 / 0.06 / 0.09 & 0.40 / 1.21 / 0.90 &  $\pm 0.8$  &  $\pm 5.0$  & 0 \\
THT & exp. & 0.97 / 0.05 / 0.53 & 0.74 / 0.29 / 2.56 &  $\pm 3.6$  &  $\pm 22.1$  & \todo{MISSING (0.04--0.09)} \\
& eff. & 0.93 / 0.06 / 0.68 & 0.31 / 0.17 / 2.58 &  $\pm 3.1$  &  $\pm 25.6$  & \todo{MISSING} \\
2-butanone & exp. & 1.60 / 0.48 / 1.99 & 3.58 / 2.85 / 2.40 &  $\pm 8.3$  &  $\pm 40.0$  & \todo{MISSING (0.3--0.6)} \\
& eff. & 0.50 / 0.40 / 1.42 & 0.78 / 1.74 / 1.57 &  $\pm 3.9$  &  $\pm 28.6$  & 0.002 \\
cyclohexanone & exp. & 2.79 / 0.92 / 2.61 & 5.82 / 1.66 / 1.92 &  $\pm 10.9$  &  $\pm 40.9$  & \todo{MISSING (0.3--0.6)} \\
& eff. & 0.60 / 0.19 / 0.82 & 0.52 / 1.93 / 2.25 &  $\pm 5.1$  &  $\pm 39.8$  & \todo{MISSING} \\
cyclopentanone & exp. & 6.31 / 5.74 / 12.0 & 6.72 / 6.21 / 3.88 &  $\pm 5.8$  &  $\pm 24.2$  & 0.64 \\
& eff. & 1.02 / 0.20 / 0.81 & 0.38 / 1.35 / 2.48 &  $\pm 4.1$  &  $\pm 33.9$  & \todo{MISSING}
\end{tabular}

```

```

\bottomrule
\end{tabular}
\end{table}

```

The two groups of compounds behave differently. For the ethers and the sulfide ($x(T_b) = 0.04$ – 0.09 , within the range where the leading-order orientational average is accurate), the experimental dipole gives vapour pressures within 0.5 – 1.0% , liquid densities within 0.05 – 0.2% and vaporisation enthalpies within 0.2 – 0.9% , no parameter reaches a bound, and freeing the dipole changes the objective by at most 13% . For the ketones ($x(T_b) \approx 0.3$ – 0.6 with realistic geometry), the experimental dipole is rejected by the data: σ hits its upper bound in every case, and the objective improves by a factor of 4 (2-butanone), 11 (cyclohexanone) and 58 (cyclopentanone) when μ_{eff} is freed, at the price of $\mu_{\text{eff}} = 0.00$, 0.32 and 0.33 D. A diagnostic run with widened bounds ($\sigma \leq 6.5$ Å, $\lambda_r \geq 6.5$) confirmed that the bound was not the cause: cyclopentanone collapsed to a single sphere ($m=1.00$, $\sigma=5.34$ Å) with deviations of $4.8 / 1.1 / 5.1\%$. Figure~\ref{fig:dev} placeholder shows the deviation patterns.

```

\begin{figure}[t]
\centering
\caption{FIGURE: results/deviations\_overview.png --- 6\times3 grid of percentage deviations
100(y^{\text{calc}}-y^{\text{exp}})/y^{\text{exp}} versus T$ for vapour pressure, liquid density and
vaporisation enthalpy of the six solvents; four SAFT-VR IMP variants (order 2/3, experimental/effective
dipole). The fixed-dipole ketone panels show systematic temperature trends; the ether and sulfide panels show
scatter around zero.}
\caption{Deviations of the four SAFT-VR IMP variants from the fitted DIPPR data.}
\label{fig:dev}
\end{figure}

```

\subsection{Isolating the modelling choices}

Three variants can be compared to isolate the effect of the polar term: SAFT-VR Mie without a polar term, SAFT-VR IMP with the experimental dipole, and SAFT-VR IMP with the effective dipole. A dedicated non-polar fit was not performed in the sessions \todo{MISSING: SAFT-VR Mie fit (order 3, $\mu=0$) for the six solvents}. Two of the effective-dipole fits, however, converged to $\mu_{\text{eff}} \approx 0$ (THP and 2-butanone), and these are numerically equivalent to a SAFT-VR Mie fit with the same bounds and objective. For THP the non-polar model reproduces the fitted data as well as the polar one ($0.49 / 0.06 / 0.09$ versus $0.51 / 0.08 / 0.20\%$), which is the expected outcome for a weakly polar molecule whose polar attraction is indistinguishable from dispersion in pure-component data; for 2-butanone the non-polar fit is much better than the fixed-dipole IMP fit, which is the signature of the truncation failure. For the ethers and sulfide, the fixed-dipole model achieves the accuracy of the free fit with one parameter fewer and with a physically grounded moment, which is the case the model was designed for. The comparison between order 2 and order 3 at fixed dipole isolates the a_3 term: it leaves the AADs essentially unchanged and halves the overshoot of T_c .

\subsection{Comparison with SAFT- γ Mie}

SAFT- γ Mie was evaluated in the third session with the group parameters shipped in the Clapeyron.jl database, on exactly the data points used for the IMP fits. All SAFT- γ Mie numbers below were computed in the conversation, not taken from the literature. The group assignments were 4-cCH $_2$ (5-ring) + cy0 for THF, 5-cCH $_2$ + cy0 for THP, CH $_3$ + CH $_2$ + CH $_3$ CO for 2-butanone, 5-cCH $_2$ + cC=O for cyclohexanone and 4-cCH $_2$ (5-ring) + cC=O for cyclopentanone; THT cannot be built because the database contains no sulfur group. Table~\ref{tab:gamma} gives the results alongside the two order-3 IMP variants.

```

\begin{table}[t]

```

```

\centering
\caption{SAFT- $\gamma$  Mie (Clapeyron group parameters, predictive) versus fitted SAFT-VR IMP (order 3) on identical DIPPR data. AAD\%,\% for  $P_{\mathrm{sat}}$  /  $\rho_L$  /  $\Delta h_{\mathrm{vap}}$ ;  $T_c$  and  $P_c$  deviations in \%.}
\label{tab:gamma}
\small
\begin{tabular}{llcr}
\toprule
Solvent & Model & AAD &  $T_c$  &  $P_c$  \\
\midrule
THF & SAFT- $\gamma$  Mie & 3.08 / 19.79 / 0.81 &  $+1.4^\circ\text{C}$  &  $-10.3^\circ\text{C}$  \\
& & & IMP, experimental &  $+0.46^\circ\text{C}$  /  $+0.22^\circ\text{C}$  /  $+0.89^\circ\text{C}$  &  $+3.1^\circ\text{C}$  &  $+19.6^\circ\text{C}$  \\
& & & IMP, effective &  $+0.44^\circ\text{C}$  /  $+0.22^\circ\text{C}$  /  $+0.87^\circ\text{C}$  &  $+2.9^\circ\text{C}$  &  $+19.1^\circ\text{C}$  \\
THP & SAFT- $\gamma$  Mie & 21.92 / 17.70 / 5.19 &  $+1.4^\circ\text{C}$  &  $-14.0^\circ\text{C}$  \\
& & & IMP, experimental &  $+0.51^\circ\text{C}$  /  $+0.08^\circ\text{C}$  /  $+0.20^\circ\text{C}$  &  $+1.6^\circ\text{C}$  &  $+7.4^\circ\text{C}$  \\
& & & IMP, effective &  $+0.49^\circ\text{C}$  /  $+0.06^\circ\text{C}$  /  $+0.09^\circ\text{C}$  &  $+0.8^\circ\text{C}$  &  $+5.0^\circ\text{C}$  \\
2-butanone & SAFT- $\gamma$  Mie & 3.15 / 0.42 / 1.04 &  $+2.8^\circ\text{C}$  &  $+22.2^\circ\text{C}$  \\
& & & IMP, experimental &  $+1.60^\circ\text{C}$  /  $+0.48^\circ\text{C}$  /  $+1.99^\circ\text{C}$  &  $+8.3^\circ\text{C}$  &  $+40.0^\circ\text{C}$  \\
& & & IMP, effective &  $+0.50^\circ\text{C}$  /  $+0.40^\circ\text{C}$  /  $+1.42^\circ\text{C}$  &  $+3.9^\circ\text{C}$  &  $+28.6^\circ\text{C}$  \\
cyclohexanone & SAFT- $\gamma$  Mie & 0.71 / 15.53 / 1.17 &  $+1.4^\circ\text{C}$  &  $+3.4^\circ\text{C}$  \\
& & & IMP, experimental &  $+2.79^\circ\text{C}$  /  $+0.92^\circ\text{C}$  /  $+2.61^\circ\text{C}$  &  $+10.9^\circ\text{C}$  &  $+40.9^\circ\text{C}$  \\
& & & IMP, effective &  $+0.60^\circ\text{C}$  /  $+0.19^\circ\text{C}$  /  $+0.82^\circ\text{C}$  &  $+5.1^\circ\text{C}$  &  $+39.8^\circ\text{C}$  \\
cyclopentanone & SAFT- $\gamma$  Mie & 86.56 / 8.06 / 31.27 &  $+28.1^\circ\text{C}$  &  $+44.5^\circ\text{C}$  \\
& & & IMP, experimental &  $+6.31^\circ\text{C}$  /  $+5.74^\circ\text{C}$  /  $+12.0^\circ\text{C}$  &  $+5.8^\circ\text{C}$  &  $+24.2^\circ\text{C}$  \\
& & & IMP, effective &  $+1.02^\circ\text{C}$  /  $+0.20^\circ\text{C}$  /  $+0.81^\circ\text{C}$  &  $+4.1^\circ\text{C}$  &  $+33.9^\circ\text{C}$  \\
THT & SAFT- $\gamma$  Mie & not representable & & \\
& & & IMP, experimental &  $+0.97^\circ\text{C}$  /  $+0.05^\circ\text{C}$  /  $+0.53^\circ\text{C}$  &  $+3.6^\circ\text{C}$  &  $+22.1^\circ\text{C}$  \\
\bottomrule
\end{tabular}
\end{table}

```

SAFT- γ Mie is accurate for 2-butanone, where it matches or beats the fixed-dipole IMP fit on density and enthalpy and gives the best critical temperature of all three models, and its vapour pressures for THF and cyclohexanone are good. Its critical temperatures are within 1.4--2.8\%,\% for four of the five compounds, closer than most IMP sets, partly because the IMP fits excluded near-critical data and partly because group parameters have been regressed to many compounds including their critical regions. Its liquid densities for the ring compounds containing the cyclic-ether (cy0) or cyclic-ketone (cC=0) groups, however, are 16--21\%,\% too low, and for cyclopentanone it fails outright. The LLM traced the failure to specific parameters rather than to the method: cyclohexane built from six ring CH₂ groups gives $P_{\mathrm{sat}} = 13.02^\circ\text{C}$ -kPa and $\rho_L = 9.16^\circ\text{C}$ -mol⁻¹ at 298.15-K, close to experiment, so the ring CH₂ group is sound; the cy0 and cC=0 entries carry no source in the database, and the dedicated five-ring cCH₂--cC=0 cross interaction ($\epsilon = 927^\circ\text{C}$ -K) drives the cyclopentanone critical temperature to 800-K against 624.5-K. Replacing it by ordinary ring CH₂ gives 9.09 / 16.21 / 2.84\%,\% and $T_c = 648^\circ\text{C}$ -K, still 16\%,\% low in density. These observations are about the state of one open database, not about SAFT- γ Mie as a method, and a literature parameterisation of these groups might perform differently \todo{CITATION NEEDED}.

Fairness of the comparison

The comparison in Table-\ref{tab:gamma} is asymmetric in three respects and should be read accordingly. First, the IMP sets were regressed to the very data on which they are evaluated, whereas SAFT- γ Mie is predictive here; low deviations for the former are expected, and the ``vs DIPPR correlation'' columns of Table-\ref{tab:aad}, which extend to $0.9T_c$ beyond the fitted density range, are the fairer measure of the IMP sets. Second, the number of adjustable parameters differs: four (or five) per compound for IMP against

zero per compound for SAFT- γ Mie, whose group parameters are transferable across many molecules. Third, the effective-dipole ketone sets are, in practice, five-parameter SAFT-VR Mie fits with the polar term nearly switched off; they describe the pure fluids but no longer carry the polar physics the model was meant to supply. The comparison therefore supports two conclusions: the fitted IMP parameters are the better choice for these pure solvents, and SAFT- γ Mie with the available database is a reliable predictive alternative only for 2-butanone.

```
\begin{figure}[t]
\centering
\caption{FIGURE: comparison/deviations\_overview.png --- deviation grid as in Fig.~\ref{fig:dev} for
SAFT- $\gamma$  Mie and the two order-3 IMP variants; the SAFT- $\gamma$  Mie density panels for THF, THP and
cyclohexanone sit 15--20\%, below zero.}
\caption{Deviations of SAFT- $\gamma$  Mie and the fitted SAFT-VR IMP sets from the same DIPPR data.}
\label{fig:gamma}
\end{figure}
```

```
\section{Limitations of LLMs in scientific work}
\label{sec:limits}
```

```
\subsection{The audit problem}
```

The central limitation observed in this study is not that the LLM produced wrong results, but that the volume and complexity of what it produced made the results hard to audit. The derivation runs to several pages of perturbation theory with density derivatives of empirical kernels; the model file is about 600 lines of Julia in which a sign error in one of a dozen similar terms would change the chain contribution by a few percent and be invisible in any single number; the fitting pipeline touches three languages, two databases and a global optimiser. All of it runs, all of it prints plausible numbers, and a wrong version would do the same. The 97 tests were written by the same LLM that wrote the code, so they share its blind spots: they can catch inconsistencies between the code and exact limits, but not a consistent misreading of the physics that survives into both. The researcher's effort therefore shifted from producing derivations and code to designing and reading verification, which is a different and in some ways more demanding skill.

```
\subsection{Other limitations observed}
```

The LLM's derivation follows the SAFT-VR Sum paper closely and, where that paper is internally inconsistent, it chose the version that reproduces SAFT-VR Mie; this is the right choice, but it was made on the basis of a numerical reduction test rather than an independent derivation from the Clausius virial theorem checked by a human. The empirical corrections χ , a_3 and γ_c were transferred from Mie fluids through the α -hypothesis, which Jervell et al.~\cite{jervell2026} showed can fail for potentials with exotic force profiles; the IMP has a single minimum and a monotone force, but this transfer was not tested against simulation. The infrastructure of the chat interface imposed real costs: state was lost between turns, long runs had to be launched detached and polled, and two deliveries were interrupted, one of which led to the LLM pasting a restructured and, as it correctly warned, untested listing. The LLM was consistently candid about what it had and had not verified, which was essential to the workflow, but that candour is itself a property to be checked rather than assumed. Finally, the model reasoned about the effective-dipole failure from the truncation of the IMP, which is correct as far as it goes, but did not raise the condensed-phase polarisation argument that a specialist would add; the LLM's physical reasoning was sound but narrower than the literature.

```
\subsection{Recommended verification practices}
```

From this experience we recommend the following practices whenever an LLM contributes derivations or code to a thermodynamic model.

```

\begin{enumerate}
\item \emph{Reduction to the parent model.} Any extension must reduce exactly, not approximately, to its
parent in the limit that removes the new physics, and the test should compare against an independent
implementation of the parent across liquid, vapour and supercritical states and across mixtures. Here the
reduction test alone verified four terms and the mixing rules simultaneously.
\item \emph{Analytic mappings.} Where a special case admits a closed-form mapping onto a known model (here,
the Keesom mapping of a dipolar Lennard-Jones fluid onto a Mie fluid), it provides a machine-precision check
of the new terms with the new physics switched on.
\item \emph{Derivatives against finite differences.} Every derivative that enters a property should be
compared with Richardson-extrapolated finite differences, including second derivatives and derivatives
through implicit solvers and quadratures; nested automatic differentiation is easy to break silently.
\item \emph{Thermodynamic identities.} The Clapeyron equation, the Euler relation for intensive properties,
and the equality of the energy and free-energy routes to  $\mathcal{C}_v$  test the physics of temperature-dependent
potentials independently of the code.
\item \emph{Independent benchmarks.} Molecular simulation of the model potential (as Muller and Gelb did
for the IMP, and Jervell et al. for Sutherland sums) is the only way to separate errors in the perturbation
theory from errors in the potential; the perturbation terms  $a_1$ -- $a_3$  can be sampled directly by Monte
Carlo of the reference fluid.
\item \emph{Re-execution of deliverables.} Listings pasted into a chat are not the files that were tested.
Every delivered artefact should be saved verbatim and rerun before use.
\item \emph{Bound diagnostics in fitting.} Parameters ending on bounds and systematic residual trends should
be reported automatically, as they were the first evidence of the ketone failure.
\end{enumerate}

```

```

\section{Conclusion}
\label{sec:conclusion}

```

We asked whether an LLM can contribute non-trivial science. The evidence from this case study is that it can. From a one-page brief and three papers, Claude Fable~5.1 produced a new SAFT equation of state with a complete derivation, an implementation that runs inside an established open-source package at a cost comparable to SAFT-VR Mie, and a verification suite that establishes exact reduction to the parent model, machine-precision agreement with analytic mappings, correct nested derivatives and thermodynamic consistency. It then built a parameter-estimation pipeline from raw database exports, fitted six compounds, diagnosed the failure of the leading-order polar term for ketones in terms of the underlying approximation, proposed and tested the standard remedy, and compared the result with a state-of-the-art group-contribution model on identical data. Along the way it found a sign inconsistency in a 2026 paper and a bug in the host package, and it caught and corrected its own errors. By the definition of Section~\ref{sec:verification}, the equation of state works.

Expert oversight remained essential, and its nature changed. The researcher did not write equations or code, but had to judge whether the verification was adequate, whether the physical conclusions were sound, and whether the candid limitations reported by the LLM were complete. The audit burden is real: we have not independently rederived the chain term, nor tested the perturbation theory against simulations of the IMP fluid, and we flag both as open.

Future work follows directly from the limitations found. The ketone failure calls for a polar term that saturates at low temperature, for example by retaining the resummed $-k_{\mathrm{B}}T\ln(1+x)$ form or a higher cumulant, rather than for effective dipoles that erase the polar physics. The perturbation theory should be tested against Monte Carlo sampling of a_1 -- a_3 for the IMP fluid. An association term is needed for hydrogen-bonding polar fluids. The fitted parameters should be tested in mixtures, where the parameter-free cross terms are the model's main promise. And the same protocol, brief, LLM, verification suite, error record, should be applied to other theories to establish whether the result reported here is

typical.

\section*{Statement on the use of AI}

The equation of state, its derivation, the Julia and Python code, the verification tests, the parameter fits and the SAFT- γ Mie comparison described in this paper were produced by Claude Fable-5.1 (Anthropic) in response to prompts from the authors, as described in Section~\ref{sec:methods}. The authors reviewed the outputs, reran the delivered code, and are responsible for the interpretation. This manuscript was drafted with the assistance of the same model from the session transcript; the authors edited and verified the text. \todo{Adjust to the journal's policy.}

\section*{Data and code availability}

The model (\texttt{SAFTVRIMP.jl}), the test suite, the in-tree patch for Clapeyron.jl, the DIPPR extraction and fitting scripts, the extracted data, the fitted parameter files, the deviation tables and the comparison scripts are available at \todo{REPOSITORY URL}. The DIPPR-801 source workbooks are proprietary and are not redistributed.

\bibliographystyle{unsrtnat}

\begin{thebibliography}{99}

\bibitem{muller2003} E.-A. M\uller, L.-D. Gelb, Ind. Eng. Chem. Res. 42, 4123 (2003).

\bibitem{lafitte2013} T.-Lafitte, A.-Apostolakou, C.-Avenda\~no, A.-Galindo, C.-S. Adjiman, E.-A. M\uller, G.-Jackson, J. Chem. Phys. 139, 154504 (2013).

\bibitem{jervell2026} V.-G. Jervell, T.-W. Maltby, A.-Aasen, M.-Hammer, \O. Wilhelmsen, J. Chem. Phys. 164, 114101 (2026).

\bibitem{walker2022} P.-J. Walker, H.-W. Yew, A.-Riedemann, Ind. Eng. Chem. Res. 61, 7130 (2022).

\bibitem{chapman1989} W.-G. Chapman, K.-E. Gubbins, G.-Jackson, M.-Radosz, Fluid Phase Equilib. 52, 31 (1989).

\bibitem{chapman1990} W.-G. Chapman, K.-E. Gubbins, G.-Jackson, M.-Radosz, Ind. Eng. Chem. Res. 29, 1709 (1990).

\bibitem{wertheim1984} M.-S. Wertheim, J. Stat. Phys. 35, 19 (1984); 35, 35 (1984).

\bibitem{wertheim1986} M.-S. Wertheim, J. Stat. Phys. 42, 459 (1986); 42, 477 (1986).

\bibitem{gross2001} J.-Gross, G.-Sadowski, Ind. Eng. Chem. Res. 40, 1244 (2001).

\bibitem{gilvillegas1997} A.-Gil-Villegas, A.-Galindo, P.-J. Whitehead, S.-J. Mills, G.-Jackson, A.-N. Burgess, J. Chem. Phys. 106, 4168 (1997).

\bibitem{barker1967} J.-A. Barker, D.-Henderson, J. Chem. Phys. 47, 4714 (1967).

\bibitem{barker1976} J.-A. Barker, D.-Henderson, Rev. Mod. Phys. 48, 587 (1976).

\bibitem{papaioannou2014} V.-Papaioannou, T.-Lafitte, C.-Avenda\~no, C.-S. Adjiman, G.-Jackson, E.-A. M\uller, A.-Galindo, J. Chem. Phys. 140, 054107 (2014).

\bibitem{macdowell2010} N.-Mac Dowell, F.-Llovel, C.-S. Adjiman, G.-Jackson, A.-Galindo, Ind. Eng. Chem. Res. 49, 1883 (2010). \todo{VERIFY}

\bibitem{avendano2011} C.-Avenda\~no, T.-Lafitte, A.-Galindo, C.-S. Adjiman, G.-Jackson, E.-A. M\uller, J. Phys. Chem. B 115, 11154 (2011).

\bibitem{muller2014} E.-A. M\uller, G.-Jackson, Annu. Rev. Chem. Biomol. Eng. 5, 405 (2014). \todo{VERIFY}

\bibitem{walker2022jcp} P.-J. Walker, T.-Zhao, A.-J. Haslam, G.-Jackson, J. Chem. Phys. 156, 154106 (2022).

\bibitem{jog1999} P.-K. Jog, W.-G. Chapman, Mol. Phys. 97, 307 (1999). \todo{VERIFY}

\bibitem{gross2006} J.-Gross, J.-Vrabec, AIChE J. 52, 1194 (2006).

\bibitem{aasen2019} A.-Aasen, M.-Hammer, \AA. Ervik, E.-A. M\uller, \O. Wilhelmsen, J. Chem. Phys. 151, 064508 (2019).

\bibitem{reveles2016} J.-Revels, M.-Lubin, T.-Papamarkou, arXiv:1607.07892 (2016).

\bibitem{mejia2022} J.-A. Mej\~ia-de-Dios, E.-Mezura-Montes, Metaheuristics.jl. \todo{VERIFY}

\bibitem{nelder1965} J.-A. Nelder, R.-Mead, Comput. J. 7, 308 (1965).

\bibitem{reed1973} T.-M. Reed, K.-E. Gubbins, Applied Statistical Mechanics (McGraw-Hill, 1973).

\bibitem{ramrattan2015} N.-S. Ramrattan, C.-Avenda\~no, E.-A. M\uller, A.-Galindo, Mol. Phys. 113, 932 (2015).

\end{thebibliography}

\appendix

\section{Derivation of SAFT-VR IMP}

This appendix reproduces the derivation produced in the first session, condensed. Energies are in kelvin;
 $x_{s,i} = z_{im_i} / \sum_j z_{jm_j}$; $\rho_s = N_{\mathrm{A}} \sum_i z_{im_i} / V$.

\paragraph{A.1 Orientational averaging.}

For pair potentials $u(r, \omega)$ whose unweighted angular average vanishes, the cumulant expansion of $\langle e^{-\beta u} \rangle_{\omega}$ gives $\Gamma \simeq -\frac{\beta}{2} \langle u^2 \rangle_{\omega}$, yielding Eq.~\ref{eq:imp}. In reduced units $\mu^* = \hat{\mu}^2 / (\epsilon \sigma^3)$, $Q^* = \hat{Q}^2 / (\epsilon \sigma^5)$, $T^* = T / \epsilon$, the contact values are $|\beta \Gamma^{\mu}(\sigma)| = \mu^* / (3T^*)$ and $|\beta \Gamma^Q(\sigma)| = 7Q^* / (5T^*)$. Because $\Gamma \propto \beta$, $\partial(\beta \Gamma) / \partial \beta = 2\Gamma$: the energy derived from the free energy is the classical Keesom energy, twice the pair free energy, which is the factor of two noted by Muller and Gelb.

\paragraph{A.2 Effective quantities.}

With $x = r/\sigma$, $s = \ln x$, the zero of u is the root of $F(s) = \ln c_1 - \lambda_1 s - \ln \sum_{n \geq 2} |c_n| e^{-\lambda_n s}$ and the minimum the root of the same form with weights $\lambda_n |c_n|$. Writing $\langle \lambda \rangle_w$ and $\mathrm{Var}_w(\lambda)$ for the mean and variance of the attractive exponents under the normalised weights $w_n e^{-\lambda_n s}$, $F'(s) = \langle \lambda \rangle_w - \lambda_1 < 0$ and $F''(s) = -\mathrm{Var}_w(\lambda) \leq 0$, so Newton converges monotonically from the Mie solution ($s=0$, resp. $s = \ln(\lambda_r/\lambda_a)/(\lambda_r - \lambda_a)$). Derivatives: $dx_{\mathrm{eff}}/dT = -(\partial u / \partial T) / (\partial u / \partial x)|_{x_{\mathrm{eff}}}$ and $d\epsilon_{\mathrm{eff}}/dT = -\partial u / \partial T|_{x_{\min}}$, with $\partial u / \partial T = \sum_n (dc_n/dT) x^{-\lambda_n}$. If a single multipolar term shares the dispersion exponent, $u = C\epsilon x^{-\lambda_r} - (C\epsilon + |c|)x^{-\lambda_a}$ is again a Mie potential with $F = 1 + |c|/(C\epsilon)$, $\sigma_m = \sigma F^{-1/(\lambda_r - \lambda_a)}$, $\epsilon_m = \epsilon F^{\lambda_r/(\lambda_r - \lambda_a)}$.

\paragraph{A.3 Reference fluid.}

$d_k(T) = \int_0^\infty \sigma_{\mathrm{eff},kk} [1 - e^{-\beta u_{kk}}] dr$; since $u(\sigma_{\mathrm{eff}}) = 0$ the moving limit contributes nothing to dd/dT . The BMCSL term per segment is $a^{\mathrm{HS}} = \frac{6}{\pi \rho_s} [(\zeta_2^3/\zeta_3^2 - \zeta_0) \ln(1 - \zeta_3) + 3\zeta_1 \zeta_2 / (1 - \zeta_3) + \zeta_2^3 / (\zeta_3(1 - \zeta_3)^2)]$ with $\zeta_{\mathrm{ell}} = \frac{\pi}{6} \rho_s \sum_{k,x} k d_k^{\mathrm{ell}}$; $\zeta_x = \frac{\pi}{6} \rho_s \sum_{k,l} x_{s,k} x_{s,l} d_{kl}^3$ and $\bar{\zeta}_x$ likewise with $\sigma_{\mathrm{eff},kl}^3$.

\paragraph{A.4 First and second order.}

$a_{1,kl} = 2\pi \rho_s \sum_n n \mathcal{I}(\lambda_n)$ with $\mathcal{I}(\lambda) = \int_{\sigma_{\mathrm{eff}}}^\infty g^{\mathrm{HS}}(\sigma/r)^\lambda r^2 dr = \int_d^\infty \sigma_{\mathrm{eff}}^\lambda$. The first piece is the SAFT-VR Mie mean-value integral $d^3 x_0^\lambda a_1^S(\lambda)$ with $a_1^S = -(\lambda - 3)^{-1} (1 - \zeta_{\mathrm{eff}}/2) / (1 - \zeta_{\mathrm{eff}})^3$; the second uses $g(r) \simeq g(d) + g'(d)(r-d)$ with Carnahan--Starling contact value and Percus--Yevick slope $d\backslash, g'(d) = -9\zeta_x(1 + \zeta_x) / [2(1 - \zeta_x)^3]$, giving $d^3 x_0^\lambda B(\lambda, x_{\mathrm{eff}})$ with $B = I_\lambda g(d) + J_\lambda d\backslash, g'(d)$, $I_\lambda = (1 - x^{3-\lambda}) / (\lambda - 3)$, $J_\lambda = [1 - (\lambda - 3)x^{4-\lambda} + (\lambda - 4)x^{3-\lambda}] / [(\lambda - 3)(\lambda - 4)]$. Hence $\mathcal{I} = -d^3 x_0^\lambda \Phi(\lambda)$. For a_2 , u^2 is a Sutherland sum with exponents $\lambda_n + \lambda_m$, and $a_2 = -\pi \rho_s k^{\mathrm{HS}} (1 + \chi) \int g^{\mathrm{HS}} u^2 r^2 dr$ gives the double sum of Section~\ref{sec:imp}, with $k^{\mathrm{HS}} = (1 - \zeta_x)^4 / (1 + 4\zeta_x + 4\zeta_x^2 - 4\zeta_x^3 + \zeta_x^4)$ and $\chi = f_1(\alpha) \bar{\zeta}_x + f_2 \bar{\zeta}_x^5 + f_3 \bar{\zeta}_x^8$.

\paragraph{A.5 Chain contact values.}

Equating the virial route $Z = 1 + 4\eta g(d) + \frac{2\pi\beta\rho}{3}\sum_n\lambda_{nc}\int_{\sigma}^{\infty}(\sigma/r)^{\lambda_n}g\,r^2dr$ with $Z = 1 + \rho\sum_n\beta^n\partial a_n/\partial\rho$ order by order in β , with $g\rightarrow g^{\text{HS}} + \beta\varepsilon g_1$ in the contact term, gives at $O(\beta)$ $g_1 = \frac{3}{2\pi\varepsilon d^3}\partial a_1/\partial\rho - \frac{1}{\varepsilon d^3}\sum_n\lambda_{nc}\mathcal{I}(\lambda_n)$, i.e. the expression of Section~\ref{sec:imp}; at $O(\beta^2)$, with the MCA $g_1(r)\simeq K^{\text{HS}}g^{\text{HS}}u/\varepsilon$ in the tail, $g_2^{\text{MCA}} = \varepsilon^{-2}\{\frac{32}{\rho}\partial K^{\text{HS}}/\partial\rho_s, S_2 + K^{\text{HS}}\partial(\rho_s S_2)/\partial\rho_s - K^{\text{HS}}\sum_{n,m}\lambda_{nc}\lambda_{mx}\theta^{\lambda_n+\lambda_m}\Phi(\lambda_n+\lambda_m)\}$, and $\gamma_c = 10[1 - \tanh(10(0.57 - \alpha))]\bar{z}_x(e^{\beta\varepsilon} - 1)e^{-6.7\bar{z}_x - 8\bar{z}_x^2}$. Density derivatives at constant T and composition follow from $\partial(\rho_s f)/\partial\rho_s = f + \bar{z}_x f'(\bar{z}_x)$.

\section{Key code excerpts}

\paragraph{Sutherland representation of the pair potential.}

```
\begin{lstlisting}[language=Julia]
function imp_sutherland(model::SAFTVRIMPModel,T,i,j)
    p = model.params
    sigA = p.sigma.values[i,j]*1e10;  eps = p.epsilon.values[i,j]
    lr = p.lambda_r.values[i,j];      la = p.lambda_a.values[i,j]
    Ce = Cl_mie(la,lr)*eps
    A6,B6,B8,B10 = imp_polar_constants(model,i,j)  # from mu, Q, alpha, m
    Tinv = 1/T; s6 = sigA^6
    c3 = -(A6 + B6*Tinv)/s6
    c4 = -B8*Tinv/(s6*sigA*sigA)
    c5 = -B10*Tinv/(s6*sigA^4)
    c = promote(Ce,-Ce,c3,c4,c5)
    lam = promote(lr,la,6.0,8.0,10.0)
    active = (true,true,!iszero(A6)&&iszero(B6)),!iszero(B8),!iszero(B10))
    return lam,c,active
end
\end{lstlisting}
```

\paragraph{Implicit solve with dual-number polishing.}

```
\begin{lstlisting}[language=Julia]
function imp_logroot(lam,w,active,s0)
    s = s0*one(w[1])
    for _ in 1:100
        ds = imp_logroot_step(lam,w,active,s);  s -= ds
        abs(ds) < 1e-12 && break
    end
    for _ in 1:3          # propagate dual parts (one order per step)
        s -= imp_logroot_step(lam,w,active,s)
    end
    return s
end
\end{lstlisting}
```

\paragraph{Core of the fused dispersion/chain pass (like pair).}

```
\begin{lstlisting}[language=Julia]
```

```

S1,dS1,L1 = _0,_0,_0
for k in 1:5
    act[k] || continue
    aS,daS = imp_aS1_fdf(lam[k],zx); B,dB = imp_B_fdf(lam[k],xeff,zx)
    t = c[k]*x0^lam[k]; Phi = aS + B
    S1 += t*Phi; dS1 += t*(daS+dB); L1 += lam[k]*t*Phi
end
# second-order double sum over unordered pairs (k,l) analogous, giving S2,dS2,L2
a1 -= w*(2*pi*rhoS*dij3*S1)
a2 += w*(pi*KHS*(1+chi)*rhoS*dij3*S2)
g1 = (L1 - 3*dS1)/eeff
g2MCA = (1.5*(rhoS_dKHS*S2 + KHS*dS2) - KHS*L2)/(eeff*eeff)
lny = log(gHS) + (tau*g1 + tau*tau*(1+gc)*g2MCA)/gHS
achain -= z[i]*(m[i]-1)*lny
\end{lstlisting}

\paragraph{Fitting objective and property methods.}
\begin{lstlisting}[language=Julia]
relative_sq_loss(pred, expt) = isfinite(pred) ? abs2((pred - expt)/expt) : one(pred)
imp_saturation_pressure(model, T) = safe_saturation(model, T)[1]
imp_saturated_liquid_density(model, T) = 1/safe_saturation(model, T)[2]
function imp_enthalpy_vaporization(model, T)
    psat, vl, vv = safe_saturation(model, T)
    isfinite(psat) || return NaN
    return VT_enthalpy_res(model, vv, T) - VT_enthalpy_res(model, vl, T)
end
estimator, objective, x0, ub, lb = Estimation(model0, toestimate(...), files;
                                         objective_form = relative_sq_loss)
x_eca, _ = Metaheuristics.optimize(objective, estimator, ECA(N=40, options=Options(iterations=150,
seed=SEED)))
x_best = polish(objective, x_eca, lb, ub) # Nelder-Mead in logistic-transformed variables
\end{lstlisting}

\end{document}

```

Notes on placeholders. Table 3 lacks the order-3 fixed-dipole cyclohexanone parameters, which the session reported only through their deviations; Table 4 lacks per-compound values of $x(T_b)$ for most rows (the transcript gives only the ranges 0.04–0.09 and 0.3–0.6 and the values for THF, THP, cyclopentanone and the 2-butanone effective fit); and a dedicated non-polar SAFT-VR Mie fit for the six solvents was never run. All can be filled from the delivered [summary.csv](#) files or a short additional run.

9/17/2026, 9:27:51 AM



Give me a pdf from this latex document

9/17/2026, 9:57:23 AM



Reviewing document layout and figure placement across pages. >

The layout looks sound at thumbnail scale. Checking the densest pages at readable resolution, the fitted-parameter and deviation tables and one figure page:

Reviewing document structure and section numbering. >

The tables render correctly. Delivering the PDF, plus the LaTeX source with its figures so you can keep editing and recompile:

Ran command, presented file >

The paper is compiled: 21 pages, no LaTeX errors. The PDF and a zip with the `.tex` file and both figures are below.

Compilation fixes (appearance only):

- The `\cite{CITATION NEEDED}` placeholders are now red `[CITATION NEEDED]` markers.
- I removed `language=Julia` from the code listings, because the `listings` package has no Julia definition.
- A few overlong lines are now allowed to break, and two long inline equations in Appendix A are now displayed equations.

Content changes, from the saved fit results:

- **Table 4, $x(T_b)$ column:** my earlier version had errors.
 - The tetrahydrothiophene value (0.086) sat in the tetrahydropyran row, whose value is actually 0.040.
 - The 2-butanone entry "0.002" was its fitted dipole moment, not x .
 - The cyclopentanone "0.64" belonged to a different geometry; its fixed-dipole value is 0.45.
 - Every cell in the column now comes from the summary files, including the ones previously marked missing. The "0.64" quoted in Section 5 is correct where it stands and is unchanged.
- **Table 3:** the missing cyclohexanone parameters (experimental dipole, order 3) are filled in from the same files.
- **Figures 1 and 2:** the two placeholders are replaced by the actual deviation plots from the fitting and comparison runs.

Still marked in red: the author list, the repository URL, the citations flagged NEEDED or VERIFY, and the non-polar SAFT-VR Mie baseline fit, which was never run.



saftvrimp_llm_paper
Pdf



saftvrimp_llm_paper_source
Zip



9/17/2026, 10:08:07 AM

